



Reversible data hiding in encrypted images using chaos theory and Chinese Remainder Theorem

Erdem Yavuz¹

Received: 27 December 2024 / Accepted: 20 May 2025 / Published online: 4 June 2025
© The Author(s) 2025

Abstract

Reversible data hiding in encrypted images (RDHEI) has recently garnered attention for embedding secret data within anonymized images, overcoming the spatial-correlation dependency of traditional RDH methods. In particular, this paper introduces a new RDHEI scheme based on chaos theory and Chinese Remainder Theorem (CRT). Initially, the host image is encrypted using a chaos-based symmetric cipher that adheres to strong cryptographic principles and demonstrates robust security features. Next, bits from the secret image are XORed with a random bit sequence generated by a chaotic function, initiated by a data-hiding key. Subsequently, a new data embedding method, based on pixel fusion and the CRT, is introduced. This data-hiding and encoding process, which incorporates two encrypted image bytes and one secret image bit, maintains a consistent embedding capacity of 0.5 bits per pixel (BPP), independent of the host image's complexity, while ensuring reversibility. At the receiving end, both the original host image and the embedded secret data can be flawlessly recovered. Furthermore, the proposed scheme enables extraction of the secret image based solely on the data-hiding key, without necessitating the decryption key, thereby achieving the method's semi-separability property. Experimental results indicate that the proposed scheme, which offers a fixed embedding capacity and induces only a minimal increase in the watermarked-to-original image size ratio (WOSR), upholds essential theoretical values necessary for encryption and withstands common cryptographic attacks. Moreover, as a fully reversible and semi-separable technique, the proposed scheme demonstrates both feasibility and efficiency in data hiding, surpassing several state-of-the-art methods in certain performance metrics.

Keywords Chaos theory · Chinese remainder theorem · Image encryption · Information security · Reversible data hiding · RDHEI

1 Introduction

In recent years, the widespread integration of information technology across nearly every sector has led to a substantial increase in the volume of data produced and transmitted over the Internet. This growth encompasses various forms of critical and sensitive information, intensifying the appeal for unauthorized access by malicious entities targeting private and personal data [1]. Consequently, safeguarding data integrity and ensuring security remain prominent challenges

within the digital landscape, as unauthorized attempts at accessing confidential information continue to pose significant risks [2].

Data hiding techniques have been applied across a diverse range of sectors, including copyright protection and digital rights management [3], covert communication [4], military and intelligence communications [5], authentication and anti-tampering [6, 7], digital forensics and anti-forensics [8], secure voting systems [9], cloud computing and storage security [10], and Internet of Things (IoT) security [11]. Over the past decade, however, a specialized form of data hiding, known as Reversible Data Hiding (RDH), has garnered considerable attention. RDH plays an essential role in areas where preserving image quality is paramount, including military operations, medical imaging, and legal documentation. It permits confidential information to be embedded within a carrier, like a cover image, while

✉ Erdem Yavuz
erdem.yavuz@btu.edu.tr; erdemyavuz29@gmail.com

¹ Department of Computer Engineering, Faculty of Engineering and Natural Sciences, Bursa Technical University, Yildirim, Bursa, Turkey

guaranteeing that the original data can be perfectly recovered after the hidden content is removed, leaving no trace of modification [12]. To put it another way, traditional data hiding techniques, such as steganography and watermarking, can cause varying degrees of permanent alteration to the cover image. However, such alterations are unsuitable for the aforementioned critical tasks that require image integrity, in which the preservation of the original image content is essential [13]. RDH addresses this issue by allowing the embedding of secret data within the cover image in a way that permits complete restoration of the original image after data extraction, thereby ensuring no permanent modification [13–16].

With recent advancements in cloud storage, users can easily outsource their multimedia files to the cloud. However, in cases where sensitive data is involved, users may demand privacy protection for their files stored on the cloud. Users often encrypt multimedia files before uploading them to a cloud server to safeguard privacy. For efficient file handling on the server, the cloud provider or system administrator might need to insert additional information into the encrypted files using secure cloud processing method. This information could include metadata such as source details, file annotations, authentication data, or timestamps, all without accessing the original content of the encrypted multimedia file [17]. The technique of embedding additional messages into encrypted media is widely utilized in cloud systems [17, 18]. For instance, when a patient's confidential medical information is encrypted to uphold privacy, a cloud administrator may add pertinent personal details to the related encrypted images. After decryption, the original content can be accurately recovered, and the additional information (e.g., patient details) can be accessed on the recipient's side. RDHEI supports this functionality, enabling seamless integration of additional information into encrypted media files without compromising original content integrity [17].

RDHEI is designed to embed supplementary data into an encrypted image while enabling both the original image and the embedded data to be recovered without loss at the receiving end. In contrast, traditional RDH algorithms are not suitable for encrypted images, as they typically rely on the inherent spatial correlations among neighboring pixels found in natural images, which facilitate efficient data embedding [12]. These algorithms require the original (cover) image to be visible to the data hiders, thereby making traditional RDH methods incompatible with applications where privacy preservation is essential. The literature offers various RDH methods, categorized by different techniques,

including histogram shifting [13, 19], interpolation [20], prediction error expansion [21, 22], etc. To overcome the limitations of classical RDH algorithms, researchers have concentrated on creating RDH techniques tailored specifically for encrypted images.

To our knowledge, Zhang's method [23] is among the earliest contributions in the field of RDHEI, encrypting the entire image with a stream cipher and embedding data by modifying a small portion of the encrypted data through a least significant bit (LSB) flipping operation. By leveraging spatial correlations preserved in the encrypted domain, the scheme enables accurate data extraction and image recovery at the receiver end. While effective for sensitive applications, its performance may diminish with highly textured images, as smaller block sizes can increase extraction errors. This approach provides a balance between security, data recovery, and computational efficiency.

One year later, building on his initial work, Zhang introduced a separable RDH scheme in a subsequent study [24] in 2012. Unlike the earlier method, which embedded data by modifying the least significant bits (LSBs) of an encrypted image and required decryption to access hidden data, this separable approach allows data extraction and image decryption independently. With this method, users can either decrypt the image or extract embedded data with the corresponding key. When both keys are available, both tasks can be performed with high fidelity. This approach improves usability in applications where privacy and separability are priorities, while also offering flexibility in data extraction and decryption accuracy. Like the previous scheme, it leverages spatial correlations for image recovery but enhances efficiency by allowing separable access based on the available key.

Building on Zhang's foundational methods, a further advancement in RDHEI is introduced by Karim and Wong through the Universal Reversible Data Embedding in Encrypted Domain (urDEED) [25]. Unlike earlier approaches that are tailored to specific media types or encryption methods, urDEED is universally applicable across any encrypted signal without requiring knowledge of the original pre-encryption features. This method achieves reversible data embedding by using Golomb–Rice coding, which encodes redundant information in the encrypted signal to create room for embedding. Notably, urDEED preserves the file size of the encrypted signal while maintaining high data capacity and reversibility, overcoming limitations in media specificity and dependency on spatial correlations. As a fully interchangeable and lossless approach, urDEED offers consistent embedding rates and high flexibility,

making it suitable for various encrypted domains, including sensitive applications like medical imaging or cloud-stored encrypted data.

In 2014, Zhang et al. [26] introduced an RDHEI scheme that leverages lossless compression (LDPC codes) to embed data within encrypted images, minimizing image distortion by modifying only a small portion of the 4th LSB. The approach enables either data extraction or image decryption independently and achieves full image recovery if both cryptographic and the data-hiding keys are available. However, the embedding capacity remains limited at this stage.

In 2015, Liao and Shu [27] introduced a RDH scheme in encrypted images, utilizing the exact average difference across several adjacent pixels to improve data security and recovery accuracy in cloud environments. Their method evaluates image block complexity by incorporating multiple neighboring pixels, addressing limitations in earlier methods that used only a few neighboring pixels or omitted border pixels. This enhanced evaluation reduces the extracted-bit error rate by selecting appropriate block sizes and embedding ratios. The method also incorporates an adjustable data embedding ratio, providing flexibility for embedding minimal data in large blocks, which is beneficial in cloud applications requiring low error rates and high data security. Experimental results indicate that the proposed scheme achieves a lower error rate in comparison to previous techniques by Zhang and Hong, particularly when using larger block sizes, thus offering a practical solution for secure data embedding and accurate data extraction in encrypted images.

Xiao et al. [28] proposed a high-capacity RDH method for encrypted images in 2017, achieving true reversibility through the use of an interpolation-based location map. Unlike previous approaches that embed data directly within encrypted pixels, this method first identifies suitable embedding locations in the original image through interpolation, generating a location map that is used for data embedding post-encryption. According to the map, the most significant bits (MSBs) of the encrypted image are modified to embed the data, enabling the receiver to extract the data precisely and recover the image. Experimental results show that this method outperforms traditional RDH schemes in payload capacity while preserving image quality and ensuring error-free recovery; however, it does not guarantee a fixed embedding capacity. The scheme is particularly well-suited for applications requiring high embedding capacity and secure data handling, such as cloud storage scenarios.

Malik et al. [17] introduced a RDH scheme for homomorphically encrypted images that employs the Paillier cryptosystem and an interpolation technique to improve privacy and data management in cloud environments. The method generates a location map for suitable pixel identification, compresses this map to replace the LSBs of border pixels, and embeds extra data into the encrypted image without direct access to the original. This approach allows for lossless retrieval of the original content and embedded data from the decrypted one while ensuring privacy and security, with experimental results showcasing its efficiency across various security key sizes. Unfortunately, their method also lacks a fixed embedding capacity, similar to the approach proposed by Xiao et al.

The paper by Bhardwaj and Aggarwal [29] proposes an enhanced separable RDH algorithm for telemedicine applications, focusing on the secure transmission of patient information. The technique leverages an Average Pixel Repetition (APR) method, converting an original image to an encrypted, scaled-up version to enable high embedding capacity without overflow or underflow issues, making it particularly suited for medical images. Data embedding is carried out on non-seed pixels in a pixel geometry matrix, ensuring that seed pixels remain unaltered to facilitate reversibility. This method supports blind extraction, allowing either data retrieval or image recovery independently when specific keys are used. Experimental results demonstrate high embedding rates and superior visual quality for both natural and medical images. The approach is highly resistant to attacks, maintaining low bit error rates and ensuring data integrity, which makes it a practical and secure choice for telemedicine frameworks. However, their method results in a watermarked-to-original image size ratio (WOSR) of 4, indicating that it increases the transmission load—measured in bytes to be transferred to the receiver's side over the Internet—by a factor of 3.

Rai et al. [12] introduced a novel RDHEI method in 2023 that utilizes a two-pass pixel value ordering (TPPVO) strategy combined with difference-preserving encryption to balance high embedding capacity and security. By preserving pixel correlations within image blocks, the method achieves high embedding capacity and visual security while ensuring reversibility. Experimental results show improved performance on smooth and complex images, though highly complex images may limit its effectiveness due to the insufficient correlation. While this method addresses previous limitations in embedding capacity and enhances performance

for cloud applications, the authors suggest that future work could focus on reducing auxiliary data overhead.

In 2024, Xiong et al. [30] proposed a high-capacity, separable, and fully RDH scheme for encrypted medical images, aiming to address limitations in interpolation technique (IT)-based RDH methods, such as simplistic encryption, low embedding rates, and non-separability. The proposed approach enhances encryption complexity by modifying the Logistic map with additional parameters and employing bit reorganization and diffusion. An enhanced interpolation technique utilizing neighboring pixel data generates a cover image for data embedding, while a novel embedding method uses the modulo-operation congruence theorem. The experimental findings reveal better performance in encryption quality, embedding capacity, and separability when compared to existing interpolation based RDH schemes. However, their method results in a WOSR of 4, indicating that it increases the transmission load again by a factor of 3.

Considering the abovementioned studies, the proposed RDHEI schemes reveal several limitations. Many methods face challenges with high embedding capacities, particularly in highly textured images, and struggle with efficiency in complex scenarios due to increased extraction errors or lack of adaptability across media types. While some approaches are separable, they often result in larger data sizes, which can significantly increase transmission load or cost, as indicated by a high WOSR metric. This makes them less efficient for applications requiring lightweight data handling. Additionally, several schemes lack fixed embedding capacities, and others depend heavily on spatial correlations of pixels, limiting their performance on complex images and reducing their generalizability across various encrypted domains. These limitations highlight the ongoing need for RDHEI methods that maximize embedding capacity, minimize load in transmission costs, ensure separability, and maintain efficiency across varied image complexities and applications. To mitigate the aforementioned limitations, this study proposes a new RDHEI scheme based on chaos theory and the CRT.

The main contributions and advantages of this study are summarized as follows:

- A new RDHEI method leveraging chaos theory and the Chinese Remainder Theorem (CRT) is proposed for information security, incorporating an innovative pixel fusion strategy for data embedding.
- The pixel fusion strategy utilizes three congruential equations (two for encrypted pixels and one for the

secret bit) to maintain a stable embedding capacity (EC) of 0.5 BPP while ensuring reversibility.

- The EC of the proposed method remains stable and is unaffected by the complexity of the host image, as it leverages the randomness of encrypted pixels rather than the redundancy of the original image pixels.
- The proposed pixel fusion embedding strategy, based on the CRT, embeds one bit of secret data using two encrypted pixels without relying on spatial correlation. This approach is broadly applicable to various media and signal formats across domains, setting it apart markedly from many other techniques in the field.
- Experimental results demonstrate that the proposed method induces minimal increase in the watermarked-to-original size ratio (WOSR) for certain image quality levels, offering greater efficiency compared to existing studies.

The remainder of this paper is structured as follows: Sect. 2 provides a detailed description of the proposed method, Sect. 3 presents the experimental findings and discussion, and Sect. 4 concludes the paper.

2 Proposed scheme

The block diagram of the proposed RDHEI scheme is illustrated in Fig. 1 and includes the main stages of image encryption and image decryption, hiding and encoding (embedding), as well as data extraction and decoding.

2.1 Image encryption and decryption

A chaotic cipher recently introduced by the author [1] is employed in the proposed scheme. Figure 2 presents the block diagram of Yavuz's symmetric chaotic cipher. As shown in the figure, this encryption method requires seven secret keys and an array of grayscale image bytes as input, producing an encrypted byte sequence as output, illustrated in Fig. 3. The chaotic cipher used in the proposed method produces encrypted images with essential cryptographic properties. Specifically, this cipher has undergone various security tests and achieved high security in terms of histogram uniformity, key sensitivity, key space, entropy, resistance to differential and noise attacks. Further details about the encryption and decryption processes can be found in the related study [1]. In the decryption stage, the encrypted image, extracted from the data extraction and decoding stage, is input into the chaotic decipher along with

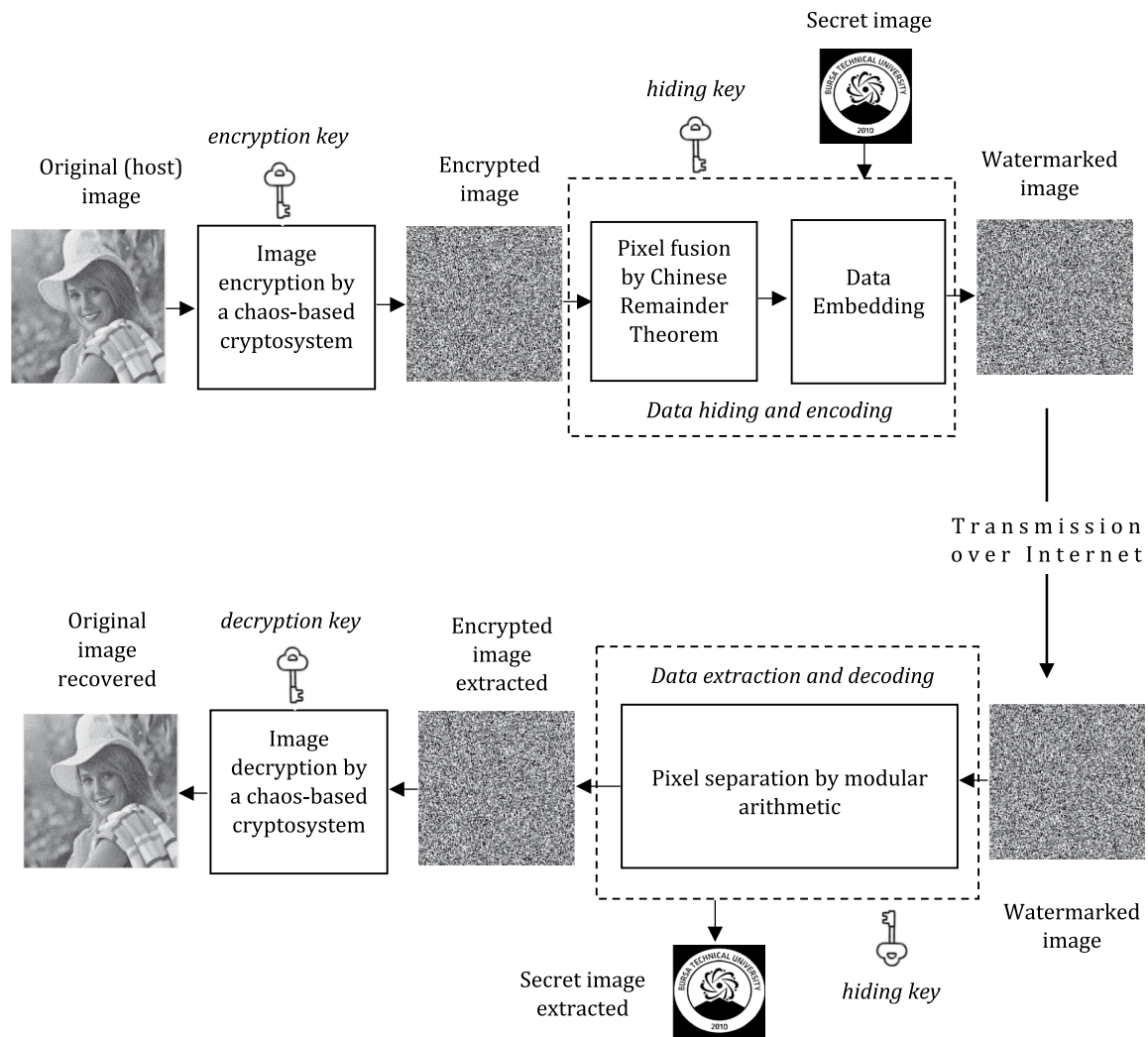
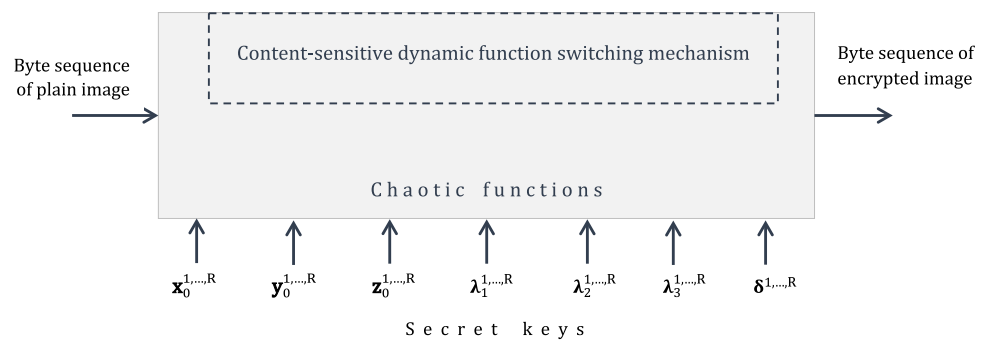


Fig. 1 Block diagram of the proposed RDH method

Fig. 2 Yavuz's chaotic cipher used in the proposed RDH method [1]



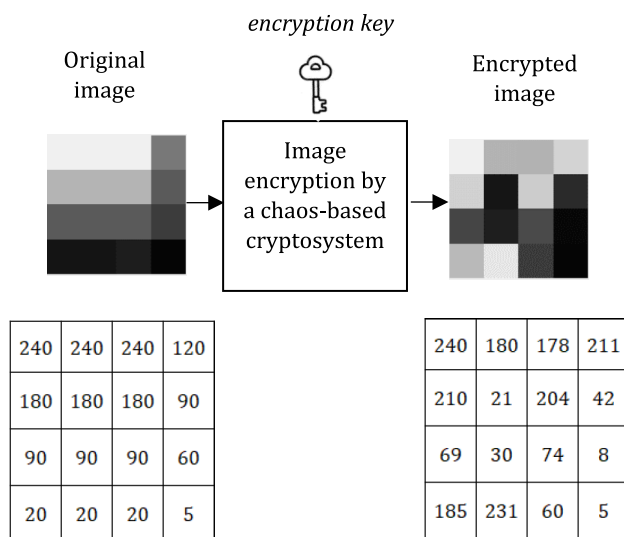


Fig. 3 Illustrative example of chaos-based encryption for a small image of size 4×4

the decryption key, resulting in the recovery of the original image.

2.2 Data hiding and encoding

This stage takes the encrypted image, secret image, and hiding key as inputs and produces a watermarked image, as

shown in Fig. 1. It comprises two processes: pixel fusion using the CRT and data embedding. An illustrative example is provided in Fig. 4, which depicts the data hiding and encoding scheme for a 4×4 image. The details of the data hiding and encoding scheme are outlined below, with its corresponding pseudocode presented in Algorithm 1.

Step 1. All pixels of the encrypted image ($W \times H$) are processed in a raster-scan fashion. In each row, two adjacent pixel bytes are grouped together in a non-overlapping manner for further processing.

Step 2. An arbitrary secret image, specifically a binary image of size $(\lfloor W/2 \rfloor \times H)$, is taken. One bit for each corresponding pair of encrypted image pixel bytes (handled in the previous step) is selected in a raster scan fashion. (The width of the secret image is half that of the encrypted image because two pixels are grouped together for processing.)

Step 3. A random bit sequence generated by a chaotic function (namely, the logistic map) is XORed one by one with the bits of the secret image data to achieve confusion. *hiding_key1* is an arbitrary fractional value between 0 and 1. A sign function is applied to the output of the chaotic function (which is also between 0 and 1) to generate binary values suitable for the XOR operation. The resultant binary value is combined with the

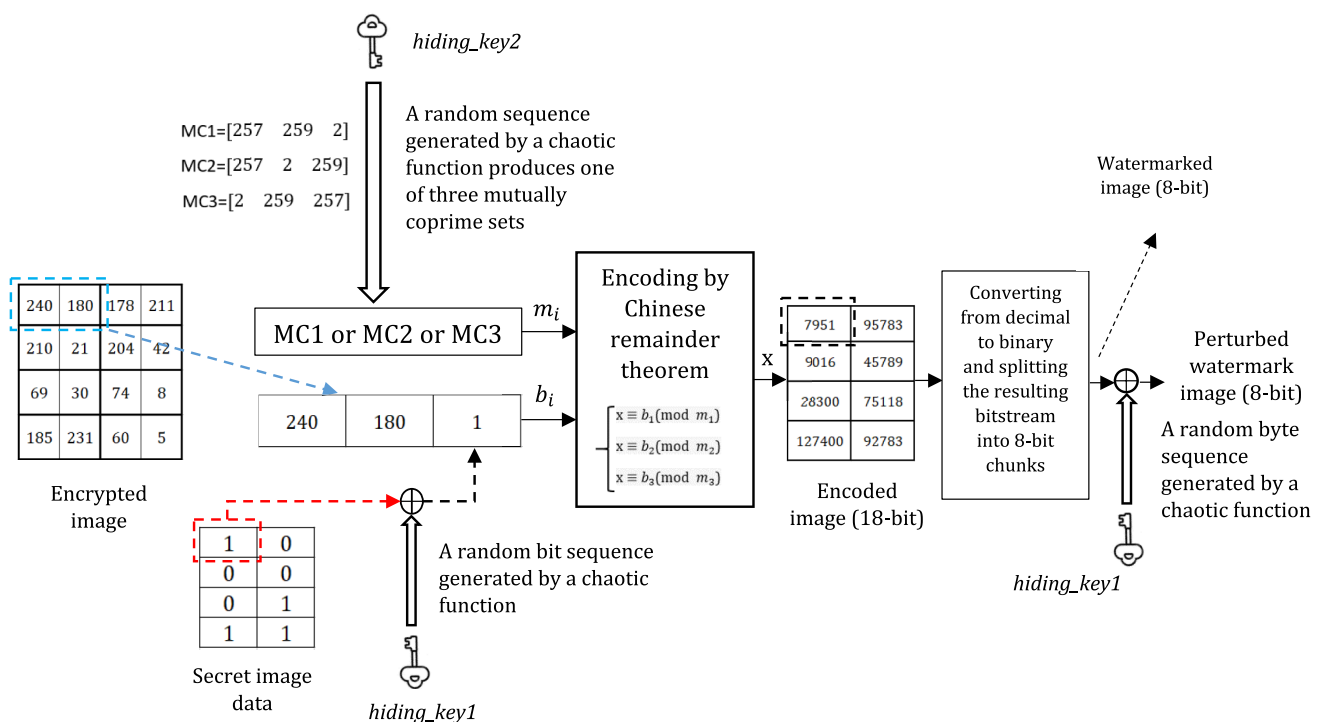


Fig. 4 An illustrative example of a data hiding and encoding scheme applied to a small image of size 4×4

encrypted pixel bytes mentioned in the previous step, forming a triplet of bytes (b_1 , b_2 and b_3).

Step 4. Another random sequence generated by a chaotic function is utilized to create one of three mutually coprime sets: MC1=[257, 259, 2], MC2=[257, 2, 259], or MC3=[2, 259, 257]. Here, we select $m_i > 255$ for encrypted pixel bytes and $m_i > 1$ for the secret bit to be embedded. The rationale for choosing these prime numbers is to ensure the generation of all possible residues up to the given value during the data extraction and decoding stage [31]. In this study, we use modulo values of 257 and 259, as they are the smallest prime numbers greater than 255—the maximum grayscale value of encrypted pixels. For embedding secret bits, we use the prime number 2, which is the smallest possible prime and suitable since secret data bits consist only of 0 and 1. Although larger primes can be chosen arbitrarily [31], increasing the prime values results in larger encoded numbers, which in turn affects the size of the watermarked image. Therefore, the coprime set is kept minimal to limit the increase in the WOSR metric. The purpose of using three variants of mutually coprime sets, as shown above, is to enable the generation of different encoded values for the same inputs (i.e., encrypted pixels and the secret bit). This approach enhances the fusion depth of the secret bit with the encrypted pixels. *hiding_key2*, an arbitrary fractional value between 0 and 1, is used to initialize the chaotic function. The chaotic function (specifically, Logistic Map) generates a pseudorandom sequence (Q_seq_j) between 0 and 1, and the selection of the coprime set is done according to the following rules:

$$m_i = \text{MC1, if } Q_seq_j > 0.667, \quad (1)$$

$$m_i = \text{MC2, else if } Q_seq_j > 0.333, \quad (2)$$

$$m_i = \text{MC3, otherwise.} \quad (3)$$

Step 5. Data encoding is performed using the CRT, which deals with congruences in number theory. The CRT states that if $m_1, m_2, \dots, m_k$ are positive integers that are mutually coprime, then M_1^{-1} is the modular

multiplicative inverse of an integer M_i modulo m_i [31]. This means that it satisfies the following equation:

$$M_i M_i^{-1} \equiv 1 \pmod{m_i}, \quad i = 1, 2, \dots, k. \quad (4)$$

For any given k numbers ($k \geq 2$): b_1 , b_2 and b_k the following congruential equations hold (in our scheme, k is equal to 3):

$$\begin{cases} x \equiv b_1 \pmod{m_1} \\ x \equiv b_2 \pmod{m_2} \\ x \equiv b_3 \pmod{m_3} \end{cases} \quad (5)$$

These equations have a unique solution modulo m :

$$x \equiv M_1 M_1^{-1} + M_2 M_2^{-1} + M_3 M_3^{-1} \pmod{m}, \quad (6)$$

where $m = m_1 \cdot m_2 \cdot m_3$. Herein, x is the encoded value of two pixel bytes and the related secret bits (b_1, b_2, b_3). From Eq. (6), we can obtain b_i by exploiting $b_i \equiv x \pmod{m_i}$, where $i = 1, 2, \dots, k$. In this way, we can successfully decode the image.

Step 6. The encoded numbers from the previous step are then combined to form the encoded image, which is the same size as the secret image data. To be precise, it is of size $[W/2] \times H$.

Step 7. The encoded image pixels (x) formed by Eq. 6 are a maximum of 18 bits long when one of the three mutually coprime sets mentioned in Step 4 is used. These 18-bit numbers are converted from decimal to binary, and the resulting bitstream is split into 8-bit chunks to create the 8-bit version of the watermarked image. (Note that the bitstream at the end is padded with zero bits to ensure 8-bit chunks.)

Step 8. A random byte sequence, generated by a chaotic function (specifically, the logistic map), is XORed with the bytes of the watermarked image, one by one, to introduce additional confusion and scrambling. This process results in a perturbed watermark image at the output, which serves as the final encoded watermarked image to be transmitted over the Internet or through any medium to the receiver.

```

Inputs:
W, H //Width and Height of the encrypted image
Encrypted_Image[W][H] // Encrypted image of size  $W \times H$ 
Secret_Image[W/2][H] //Secret binary image of size  $(W/2) \times H$ 
hiding_key1 // Fractional value between 0 and 1 (used for chaotic sequence)
hiding_key2 // Fractional value between 0 and 1 (used to select coprime set)
Outputs:
Watermarked_Image // Watermarked image, size of which is slightly  $> W \times H$ 
Begin
// Step 1: Group two adjacent pixel bytes
for j from 1 to H do
  for i from 0 to (W/2) do
    b1  $\leftarrow$  Encrypted_Image[i][j]
    b2  $\leftarrow$  Encrypted_Image[i+1][j]

// Step 2: Select one bit from the secret image for each pixel pair
for j from 1 to H do
  for i from 1 to (W/2) do
    secret_bits  $\leftarrow$  Secret_Image[i][j]

// Step 3: Generate chaotic random bit sequence using logistic map and XOR
for j from 1 to H do
  for i from 1 to (W/2) do
    chaotic_value  $\leftarrow$  Logistic_Map(hiding_key1)
    binary_chaotic_bits  $\leftarrow$  Sign(chaotic_value)
    // XOR operation with secret bit
    b3  $\leftarrow$  secret_bits XOR binary_chaotic_bits

// Step 4: Select coprime set based on hiding_key2
Q_seq  $\leftarrow$  Logistic_Map(hiding_key2)
if Q_seq > 0.667 then
  coprime_set  $\leftarrow$  [257, 259, 2] // MC1
else if Q_seq > 0.333 then
  coprime_set  $\leftarrow$  [257, 2, 259] // MC2
else
  coprime_set  $\leftarrow$  [2, 259, 257] // MC3

// Step 5: Perform encoding using CRT
m1  $\leftarrow$  coprime_set[0]
m2  $\leftarrow$  coprime_set[1]
m3  $\leftarrow$  coprime_set[2]

M  $\leftarrow$  m1 * m2 * m3
M1  $\leftarrow$  M / m1
M2  $\leftarrow$  M / m2
M3  $\leftarrow$  M / m3

M1_inv  $\leftarrow$  ModInv(M1, m1)
M2_inv  $\leftarrow$  ModInv(M2, m2)
M3_inv  $\leftarrow$  ModInv(M3, m3)

x  $\leftarrow$  (b1 * M1 * M1_inv + b2 * M2 * M2_inv + b3 * M3 * M3_inv) mod M
// Step 6: Combine encoded numbers to form the resultant image
Encoded_Image[i][j]  $\leftarrow$  x // Store encoded values

// Step 7: Convert 18-bit encoded image pixels to 8-bit format
Bitstream  $\leftarrow$  []
for j from 1 to H do
  for i from 1 to W/2 do
    x  $\leftarrow$  Encoded_Image[i][j]
    Bitstream  $\leftarrow$  Bitstream + Dec2Bin(x, 18)

// Pad bitstream to ensure 8-bit chunks
while length(Bitstream) mod 8  $\neq$  0 do
  Bitstream  $\leftarrow$  Bitstream + "0"

// Generate 8-bit watermarked image
for i from 1 to length (Bitstream)/8 do
  Watermarked_Image[i mod (W/2)][i div (W/2)]  $\leftarrow$  BinToDec(Bitstream[8*(i-1)+1 : 8*i])

// Step 8: XOR final output for additional confusion
[i, j]  $\leftarrow$  sizeof(Watermarked_Image)
for j from 1 to H do
  for i from 1 to W do
    chaotic_byte  $\leftarrow$  Logistic_Map(hiding_key1)
    Watermarked_Image[i][j]  $\leftarrow$  Watermarked_Image[i][j] XOR chaotic_byte
End

```

Algorithm 1 Data hiding and encoding

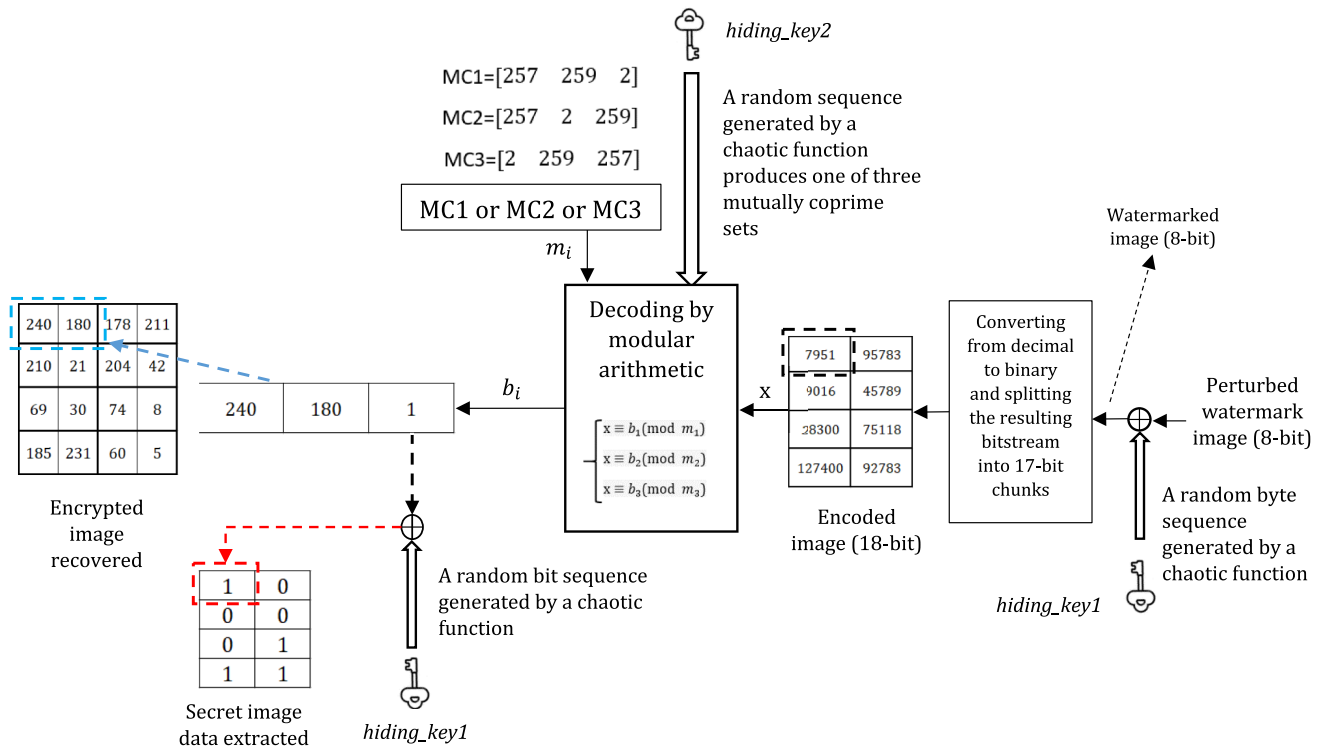


Fig. 5 An illustrative example of a data extraction and decoding scheme for a small image of size 4×4

2.3 Data extraction and decoding

This stage takes the watermarked image along with the hiding key as inputs and extracts the encrypted image and secret image, as seen in the lower part of Fig. 1. Data extraction and decoding are realized by performing modular arithmetic on the elements of the watermarked image. An illustrative example is provided in Fig. 5, depicting the data extraction and decoding scheme for a small image of size 4×4 . The details of this scheme are outlined below, with its corresponding pseudocode presented in Algorithm 2.

- Step 1.** The received encoded watermarked image is XORed, byte by byte, with a random byte sequence generated by a chaotic function (specifically, the logistic map) to reverse the related additional confusion and scrambling applied in the final step of the embedding stage. This process produces the unperturbed watermark image.
- Step 2.** The received watermarked image pixels (transmitted over the Internet or any other medium), consisting of 8-bit numbers, are converted from decimal to binary. The resulting bitstream is then split into 18-bit chunks to create the encoded image version, with pixels that are 18 bits long. (Note that if the bitstream was padded with zero bits before to ensure 8-bit chunks, they are discarded.)

- Step 3.** All elements of the encoded image ($[W/2] \times H$) are processed in a raster scan fashion, and each element x , which is the value to be decoded, is handled one by one. In fact, this value is the encoded/fused version of two encrypted pixel bytes and one secret image bit.
- Step 4.** The same random bit sequence generated by a chaotic function described in Step 4 of the data hiding and encoding stage is utilized to generate one of three mutually coprime sets: $MC1 = [257, 259, 2]$, $MC2 = [257, 2, 259]$, or $MC3 = [2, 259, 257]$. The variable *hiding_key2*, which is the same arbitrary fractional value between 0 and 1 used in the data embedding stage, is utilized to initialize the chaotic function. This chaotic function generates a pseudorandom sequence (Q_seq_j) between 0 and 1, and the selection of the coprime set is made according to the aforementioned rules given in Eqs. (1–3).
- Step 5.** Equation (5) is utilized to obtain the decoded values (b_1, b_2, b_3), which represent two encrypted pixel bytes and the associated secret bit. In this process, decoding is performed using modular arithmetic with one of the mutually coprime sets mentioned in the previous step.
- Step 6.** Another random bit sequence generated by a chaotic function (the same one used in Step 3 of the data embedding stage) is XORed one by one with the decoded bit value (obtained in the previous step) to recover secret image bits. The variable *hiding_key1* is an arbitrary fractional value between 0 and 1. A sign function is

applied to the output of the chaotic function (which is also between 0 and 1) to generate binary values suitable for the XOR operation.

Step 7. The resultant binary values are combined to form the extracted secret binary image. Each binary value is obtained from one of the three remainder terms (b_i). The chaotic function from the previous step determines which of these three remainder terms, the same as those

used in the data hiding and encoding stage, corresponds to the one used to obtain the decoded secret bit.

Step 8. The other two remainder terms (b_i) hold two pixel bytes of the encrypted image. These are combined in a raster scan fashion to reconstruct the extracted encrypted image.



Fig. 6 Several test images: **a** Clock; **b** Airplane; **c** Lena; **d** Elaine; **e** Peppers; **f** Gray21LevelStepWedge; **g** Male; **h** Airport; **i** Custom image kid Alper

```

Inputs:
W, H // Width and Height of the encrypted image
Watermarked_Image // Received watermarked image
hiding_key1 // Fractional value between 0 and 1 (chaotic sequence for XOR)
hiding_key2 // Fractional value between 0 and 1 (to select coprime set)

Outputs:
Secret_Image_Extracted[W/2][H] // Extracted secret binary image
Encrypted_Image_Extracted [W][H] // Extracted encrypted image

Begin
// Step 1: Reverse scrambling using XOR
[M, N] = sizeof(Watermarked_Image)
for j from 1 to M do
  for i from 1 to N do
    chaotic_byte ← Logistic_Map(hiding_key2)
    Unperturbed_Watermark[i][j] ← Watermarked_Image[i][j] XOR chaotic_byte

// Step 2: Convert to binary and split into 18-bit chunks
Bitstream ← []
for j from 1 to M do
  for i from 1 to N do
    Bitstream ← Bitstream + Dec2Bin(Unperturbed_Watermark[i][j], 8)

// Discard trailing padding bits if the bitstream was previously padded
while length(Bitstream) mod 8 ≠ 0 do
  Bitstream ← Bitstream[1 : length(Bitstream)-1]
Encoded_Image ← Split(Bitstream, 18)

// Step 3: Process each element in raster scan order
k ← 0
for j from 1 to H do
  for i from 1 to W/2 do
    k ← k + 1
    x ← Encoded_Image[i][j]

// Step 4: Select coprime set based on hiding_key2
Q_seq ← Logistic_Map(hiding_key2)
if Q_seq > 0.667 then
  coprime_set ← [257, 259, 2] // MC1
else if Q_seq > 0.333 then
  coprime_set ← [257, 2, 259] // MC2
else
  coprime_set ← [2, 259, 257] // MC3

// Step 5: Perform modular decoding
m1 ← coprime_set[0]
m2 ← coprime_set[1]
m3 ← coprime_set[2]

b1 ← x mod m1
b2 ← x mod m2
b3 ← x mod m3

// Step 6: XOR decoded secret bits with chaotic sequence
chaotic_value ← Logistic_Map(hiding_key1)
secret_bits[k] ← b3 XOR Sign(chaotic_value)

k ← 0
for j from 1 to H do
  for i from 1 to W/2 do
    k ← k + 1
    // Step 7: Reconstruct secret binary image
    Secret_Image_Extracted [i][j] ← secret_bits[k]

    // Step 8: Reconstruct encrypted images
    Encrypted_Image_Extracted [i*2-1][j] ← [b1]
    Encrypted_Image_Extracted [i*2][j] ← [b2]

End

```

Algorithm 2 Data extraction and decoding

Table 1 Global entropy values of original (plaintext) and watermarked cipher images

Test image #	Test image	Size	Global entropy of original image	Global entropy of watermarked cipher image
1	Clock	256 × 256	6.7056	7.9968
2	Airplane	256 × 256	6.4522	7.9973
3	Lena	512 × 512	7.4455	7.9994
4	Elaine	512 × 512	7.5059	7.9993
5	Peppers	512 × 512	7.5936	7.9993
6	Gray21LevelStepWedge	512 × 512	4.3922	7.9994
7	Male	1024 × 1024	7.5237	7.9998
8	Airport	1024 × 1024	6.8303	7.9998
9	Custom photo kid Alper	1024 × 1024	7.5959	7.9998

3 Experimental results and discussion

The proposed method is implemented in MATLAB for ease of plotting and visualization, and subsequently converted to C using MATLAB's code conversion and generation utility to measure more accurate execution times. The program is executed on a 64-bit Windows OS running on a personal computer (PC) equipped with an Intel® Core i5-8250U CPU @ 1.80 GHz and 8 GB of RAM. For the experiment, we selected eight standard grayscale images (available from a well-known database [32]) and one custom image, as displayed in Fig. 6. These images underwent standard tests, with detailed explanations and discussions presented in the following subsections.

3.1 Performance analysis of encryption method

This subsection presents analyses assessing the performance and robustness of the proposed scheme in terms of encryption quality, including information entropy, correlation analysis, and resistance to differential attacks.

3.1.1 Information entropy analysis

Entropy in a system is commonly viewed as a measure for quantifying the system's level of disorder. Specifically, it reflects the amount of information needed to describe all possible states of the system [33]. For grayscale images, information entropy assesses the spread of intensity values throughout the image. Higher entropy values indicate a more even distribution of these intensity levels [2, 34].

The entropy is mathematically expressed as:

$$H(m) = \sum_{i=0}^{2^N-1} p(m_i) \log_2 \frac{1}{p(m_i)}. \quad (7)$$

Table 2 Local Shannon entropy test results for watermarked cipher images ($k=30$, $T_B=1936$, $\alpha=0.001$)

Test image #	Watermarked cipher image of the test image	Local entropy value
1	Clock	7.9017
2	Airplane	7.9034
3	Lena	7.9025
4	Elaine	7.9017
5	Peppers	7.9019
6	Gray21LevelStepWedge	7.9019
7	Male	7.9034
8	Airport	7.9023
9	Custom photo kid Alper	7.9022

Here, $p(m_i)$ denotes the probability of the symbol m_i , and N represents the total number of intensity levels. For 8-bit grayscale images, there are 256 unique intensity levels. In an ideal image with a perfectly uniform distribution of intensity values, each level has an equal probability, resulting in the maximum entropy value of 8. Maximum entropy reflects the greatest possible level of randomness or disorder among the image pixels [2]. Table 1 displays the global entropy values for both the original and watermarked cipher images, corresponding to the test images shown in Fig. 6. The global entropy results confirm that the proposed scheme generates highly randomized cipher images, with entropy values nearing the theoretical maximum of 8.

The aforementioned global Shannon entropy measure alone is insufficient to establish true randomness. In some instances, relying solely on this metric can be misleading. For example, two images—one visually perceivable and the other resembling random noise—may exhibit similarly high global Shannon entropy values [1, 35]. Therefore, high global entropy does not necessarily guarantee true randomness. To more accurately assess the randomness of cipher images, the local Shannon entropy test is employed. This test involves calculating the average entropy across several

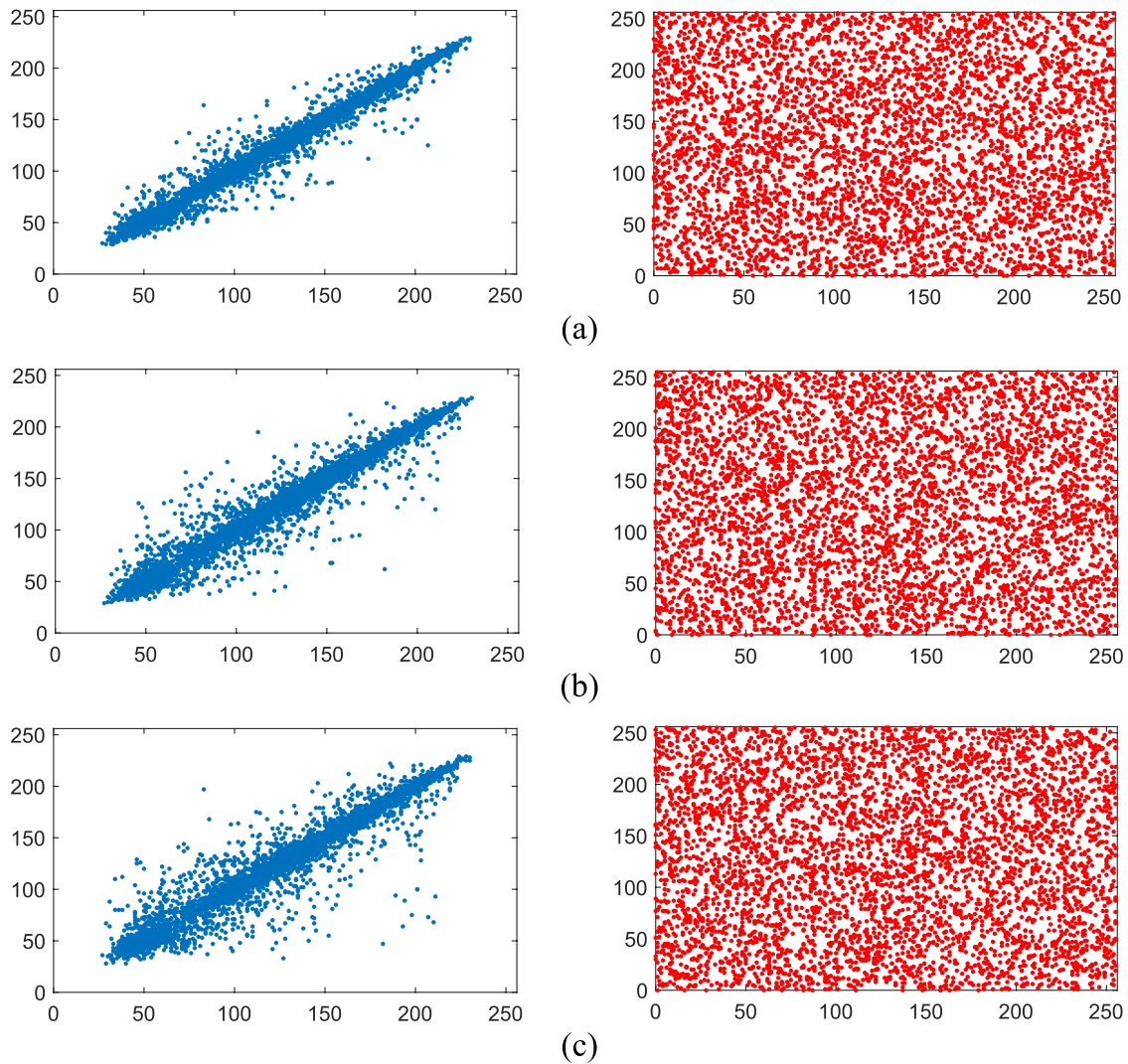


Fig. 7 Adjacent pixel correlations at (a) horizontal, (b) vertical and (c) diagonal orientations for the test image “Lena” and its cipher counterpart after embedding (watermarked)

randomly selected, non-overlapping blocks, yielding the Local Shannon entropy metric [36], as defined below.

$$\overline{H}_{k,T_B}(S) = \sum_{i=1}^k \frac{H(S_i)}{k}. \quad (8)$$

In this context, T_B refers to the number of pixels in each local block, while k denotes the number of blocks selected at random, represented by S_i . The ideal local entropy value is determined by the parameters T_B , k , and the significance level α . Readers interested in further details may consult

Table 3 Correlation coefficients of the original test images and corresponding watermarked cipher images (cipher images after embedding)

Test image	Original image			Encrypted image (after embedding)		
	Horizontal	Vertical	Diagonal	Horizontal	Vertical	Diagonal
Clock	0.9752	0.9572	0.9377	−0.0123	0.0002	−0.0050
Airplane	0.9114	0.9612	0.8662	−0.0202	0.0029	0.0058
Lena	0.9842	0.9713	0.9564	−0.0050	−0.0083	0.0044
Elaine	0.9727	0.9758	0.9686	0.0048	0.0065	0.0048
Peppers	0.9770	0.9732	0.9595	0.0209	0.0179	0.0086
Gray21LevelStepWedge	0.9998	0.9959	0.9957	0.0280	−0.0076	−0.0068
Male	0.9797	0.9805	0.9653	−0.0172	0.0112	−0.0153
Airport	0.9019	0.9014	0.8408	0.0246	−0.0059	−0.0875
Custom photo kid Alper	0.9858	0.9824	0.9721	−0.0255	0.0038	−0.0075

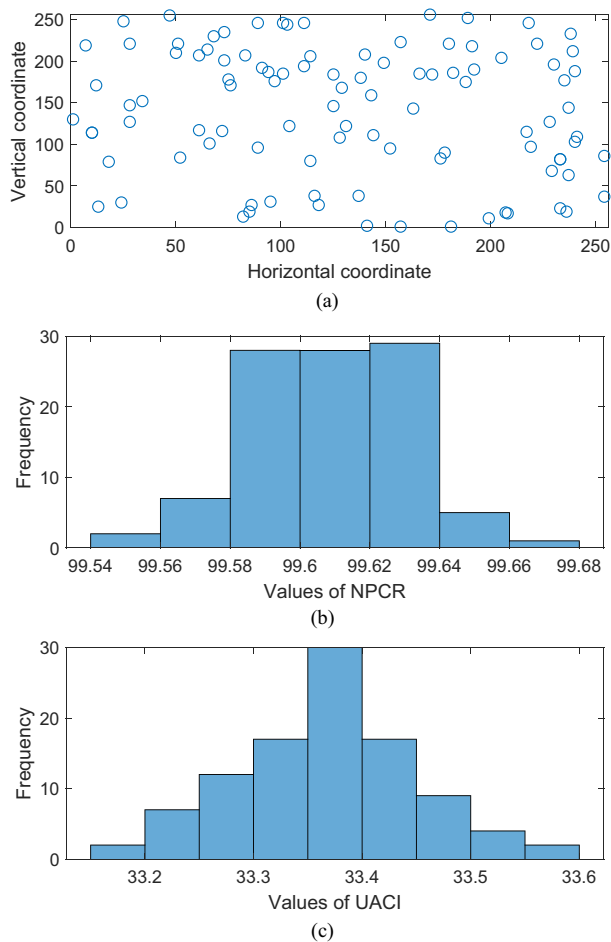


Fig. 8 Results from 100 experiments: **a** Distribution of randomly selected pixel locations, **b** histogram of NPCR values, and **c** histogram of UACI values

Table 4 Results of reversibility analysis

Test image #	Name of the test image	PSNR _{rec} ^{org} (in dB)	PSNR _{ext} ^{emb} (in dB)
1	Clock	+∞	+∞
2	Airplane	+∞	+∞
3	Lena	+∞	+∞
4	Elaine	+∞	+∞
5	Peppers	+∞	+∞
6	Gray21LevelStepWedge	+∞	+∞
7	Male	+∞	+∞
8	Airport	+∞	+∞
9	Custom photo kid Alper	+∞	+∞

the comprehensive study [36], which outlines the theoretical mean and standard deviation for ideal local Shannon entropy values across various block sizes. According to reference study, a cipher image passes the test if its average local Shannon entropy values lie between the critical thresholds $[h_{left}^*, h_{right}^*]$ [35, 36]. Following the guidelines of the reference study [36], the parameter k was set to 30, which

Fig. 9 A meaningful binary image of size 512×256 (secret image)



represents the minimum number of non-overlapping image blocks required for analysis [35]. Additionally, the parameter T_B was optimally set to 1936 (44×44) pixels for the experiment. Table 2 presents the local Shannon entropy values for watermarked cipher images based on these parameters. Since all the local entropy results fall within the critical range $[h_{left}^* = 7.901515698, h_{right}^* = 7.903422936]$ for a significance level of $\alpha = 0.001$, it can be conclusively stated that the proposed scheme has successfully passed the test [35]. (Note that the watermarked cipher image is obtained by embedding a properly sized version of the secret image, which is the logo shown in Fig. 9)

3.1.2 Correlation analysis

In natural images, neighboring pixels typically exhibit similar intensity values, resulting in high correlations between adjacent pixels in horizontal, vertical, and diagonal directions [35, 37]. Disrupting these strong correlations is essential to prevent potential security vulnerabilities, as such patterns can facilitate attacks [1]. To evaluate and illustrate the strength of these correlations, we randomly sample 5,000 pixels from the image and plot the correlations between adjacent pixels across three orientations: horizontal, vertical, and diagonal. The clustering of pixel pairs along the first diagonal (i.e., $y=x$ line) in the first-column subplots of Fig. 7 demonstrates the high correlations in original test images (namely, Lena) at these orientations. In contrast, the dispersed pixel distributions in the second-column subplots of Fig. 7 show the disrupted correlations in the watermarked images (cipher counterpart after embedding) produced by the proposed scheme, visually affirming the decorrelation of adjacent pixels.

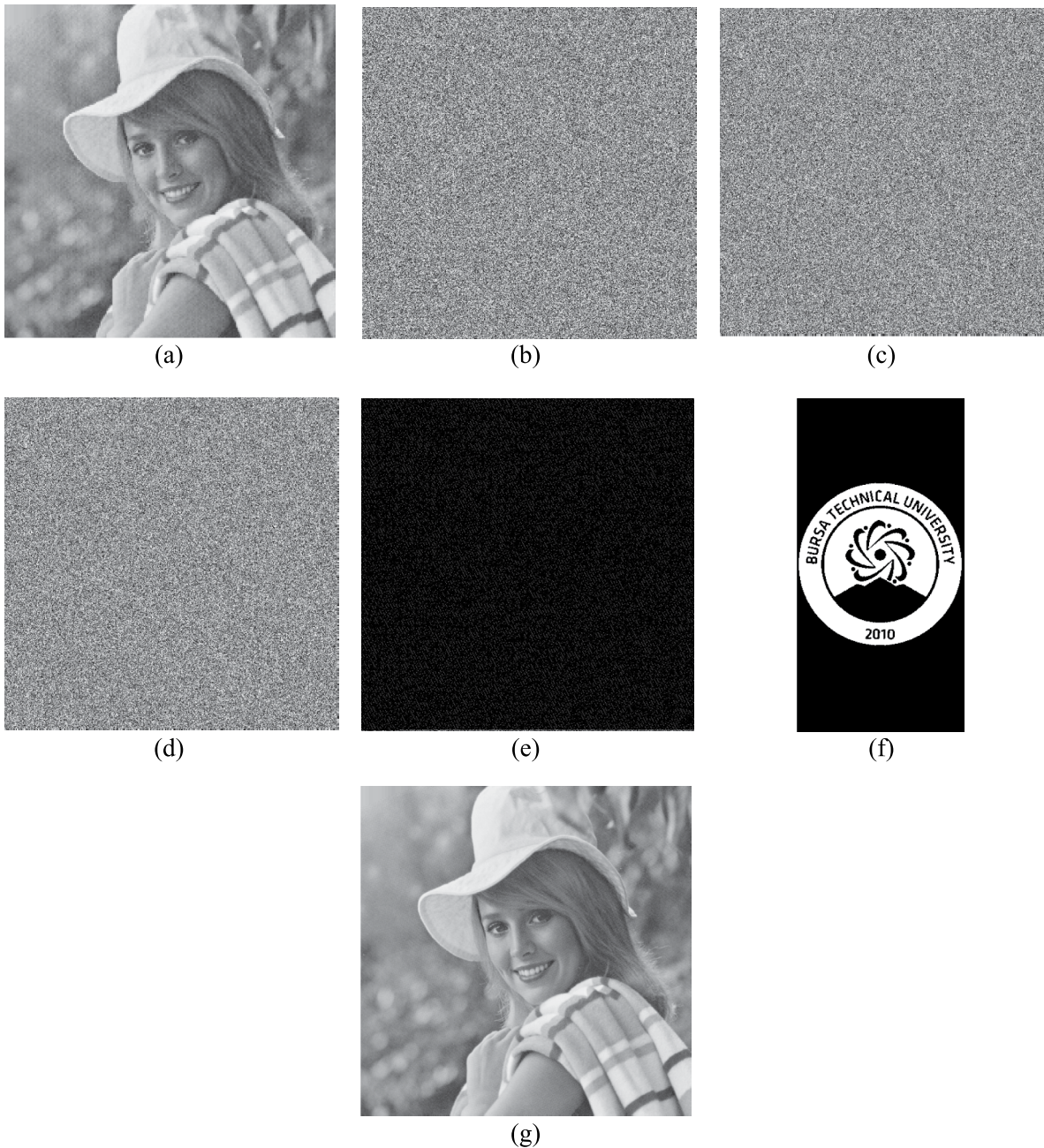


Fig. 10 Experiment on image Elaine: **a** Original image; **b** Encrypted image; **c** Encoded cover image; **d** Watermarked image; **e** Absolute difference of encoded image and watermarked image; **f** Extracted secret image data with zero error rate; **g** Recovered image, the same as original image

Quantitative measures are also applied in cryptosystems to assess correlation. A commonly used metric is the correlation coefficient, which quantifies the relationship between adjacent pixels across different directions and is calculated as follows:

$$\text{corr}(x, y) = \frac{E[(x - \mu_x)(y - \mu_y)]}{\sigma_x \sigma_y}. \quad (9)$$

Here, x and y represent two data arrays containing the intensities of neighboring pixels, while $E[.]$ denotes the expectation operator. In this context, μ_x and μ_y refer to the mean values of sequences x and y , and σ_x and σ_y denote their respective standard deviations. When the correlation coefficient approaches 1, it signifies a strong relationship between adjacent pixels, while a value closer to zero implies the absence of correlation [35]. Table 3 presents the correlation coefficients for the original test images and the

Table 5 PSNR between watermarked and encoded images

Test image #	Name of the test image	PSNR _{wat} ^{enc}
1	Clock	24.2020
2	Airplane	24.2020
3	Lena	24.2047
4	Elaine	24.2047
5	Peppers	24.2047
6	Gray21LevelStepWedge	24.2047
7	Male	24.1887
8	Airport	24.1887
9	Custom photo kid Alper	24.1887

corresponding watermarked cipher images (cipher images after embedding). The findings show that the encrypted images generated by the proposed scheme exhibit minimal correlation between adjacent pixels, which reflects the scheme's success in breaking the pixel correlations present in the original images.

3.1.3 Differential attack analysis

The number of pixel change rate (NPCR) and unified average changing intensity (UACI) are widely utilized metrics

for evaluating a cryptosystem's resilience against differential attacks [35, 38, 39]. In this type of attack, an original image and a minimally altered version of it—often differing by only a single bit or a single byte—are used as inputs for the cryptosystem under evaluation. By closely examining the similarities and variations in the resulting ciphered images, an attacker aims to identify vulnerabilities within the system. The NPCR and UACI metrics provide a quantitative measure of these differences and are computed as follows:

$$\text{NPCR} = \frac{\text{Diff}(C_1, C_2)}{W.H} \cdot 100\%, \quad (10)$$

$$\text{UACI} = \frac{1}{W.H} \sum_{i=0}^{W-1} \sum_{j=0}^{H-1} \frac{|C_1(i, j) - C_2(i, j)|}{255} \cdot 100\%. \quad (11)$$

In this context, C_1 and C_2 represent the ciphered versions of the original image and its slightly modified counterpart, respectively, while W and H denote the width and height of the image. First, the original image is encrypted along with

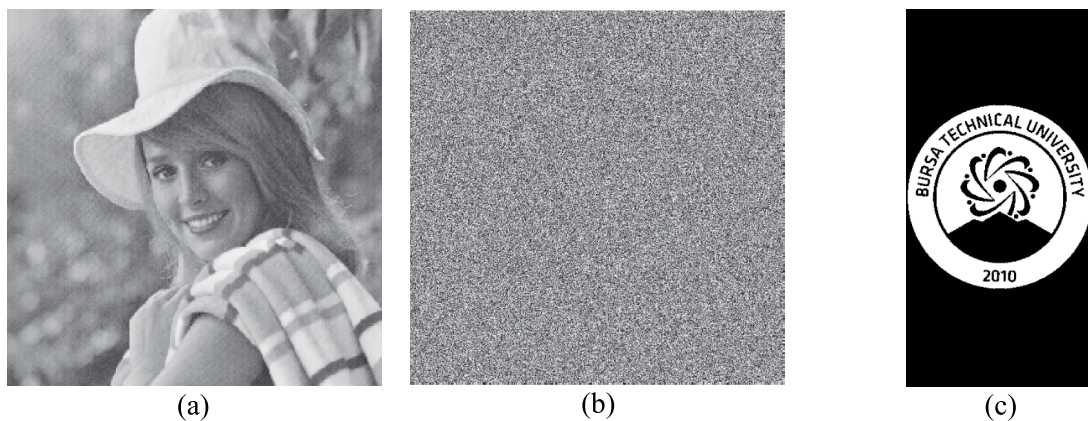


Fig. 11 Experimental results while having only data-hiding key: **a** Original image; **b** Recovered noise-like cover image; **c** Extracted secret image

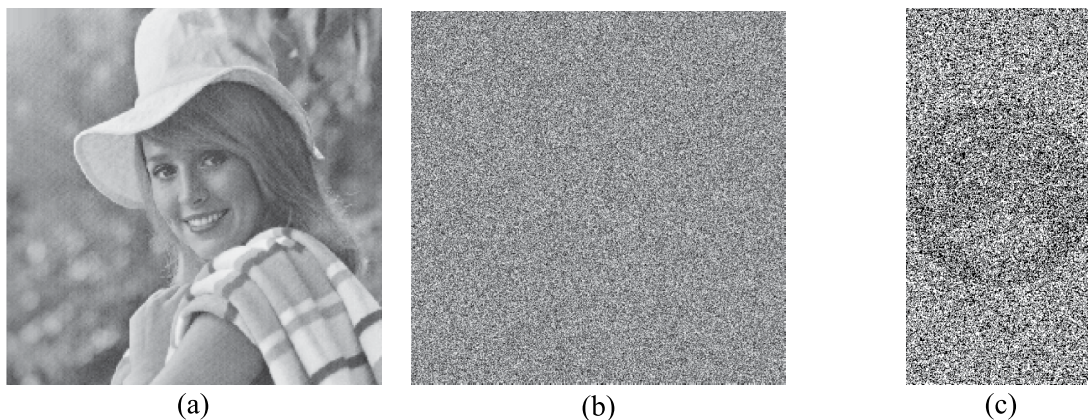


Fig. 12 Experimental results while having only decryption key: **a** Original image; **b** Recovered noise-like cover image; **c** Extracted noise-like secret image

Table 6 Comparison of the size (in bits) of watermarked images generated by different schemes

Schemes	Original image	Encrypted image	Watermarked image
Xiao et al. [28]	$W \times H \times 8$	$W \times H \times 8$	$W \times H \times 8$
Xiong et al. [30]	$W \times H \times 8$	$W \times H \times 8 \times 4$	$W \times H \times 8 \times 4$
Bhardwaj and Aggarwal [29]	$W \times H \times 8$	$W \times H \times 8 \times 4$	$W \times H \times 8 \times 4$
Malik et al. [17]	$W \times H \times 8$	$W \times H \times (\lfloor \log_2 N^2 \rfloor + 1)$	$W \times H \times (\lfloor \log_2 N^2 \rfloor + 1)$
Zhang et al. [40]	$W \times H \times 8$	$W \times H \times (\lfloor \log_2 N^2 \rfloor + 1)$	$W \times H \times (\lfloor \log_2 N^2 \rfloor + 1)$
Bhardwaj [20]	$W \times H \times 8$	$W \times H \times 4 \times (\lfloor \log_2 N^2 \rfloor + 1)$	$W \times H \times 4 \times (\lfloor \log_2 N^2 \rfloor + 1)$
Proposed scheme	$W \times H \times 8$	$W \times H \times (8 + 1)$	$W \times H \times (8 + 1)$

Table 7 Features of the evaluated schemes

Schemes	Encryption	Key	Embedding	Extraction	Secret data	Original image
Xiao et al. [28]	XOR	Symmetric	Relationship	Non-separable	Yes	Yes
Xiong et al. [30]	Chaotic	Symmetric	Addition	Separable	Yes	Yes
Bhardwaj and Aggarwal [29]	XOR	Symmetric	Addition	Separable	Yes	Yes
Malik et al. [17]	Paillier	Asymmetric	Relationship	Non-separable	Yes	Yes
Zhang et al. [40]	Paillier	Asymmetric	Addition	Separable	No	Yes
Bhardwaj [20]	Paillier	Asymmetric	Relationship	Non-separable	Yes	Yes
Proposed scheme	Chaotic	Symmetric	CRT	Semi-separable	Yes	Yes

a secret image to produce the encrypted (cipher) image, also referred to as the watermarked image. Subsequently, a pixel in the original image is randomly selected, and its value is inverted to introduce a slight modification in it. These two versions are then encrypted alongside the secret image (a properly sized version of the image shown in Fig. 9). Finally, for the two watermarked images, Eqs. (10) and (11) are applied to compute the NPCR and UACI values. The procedure is carried out 100 times [30], and the resulting experimental data is shown in Fig. 8. In their comprehensive study on randomness tests for NPCR and UACI, Wu et al. [34] identified the critical values [35] for ideally encrypted images of various sizes. With reference to their study, as shown in the figure, the NPCR and UACI values obtained from the 100 experiments closely approximate the respective theoretical values. Specifically, for a cipher image pair of size 256×256 , if the NPCR score exceeds the threshold $N_{0.001}^* = 99.5341\%$, the image pair can be considered random-like at a significance level of $\alpha = 0.001$. The theoretical UACI critical values comprise an interval with lower and upper bounds ($u_{0.001}^{*-} = 33.1594\%$, $u_{0.001}^{*+} = 33.7677\%$) for this significance level [34]. To pass the UACI test, the score for a cipher image pair must fall within these bounds. Accordingly, our scheme demonstrates high plaintext sensitivity and robust resistance to differential attacks.

3.2 Performance analysis of data embedding method

In this section, two widely used quantitative metrics, Peak Signal-to-Noise Ratio (PSNR) and Bits Per Pixel (BPP), are employed to objectively assess the image quality and data embedding capacity (EC) of the RDH scheme. These metrics are formally defined as follows:

$$\text{PSNR} = 10 \cdot \log_{10}(255/\text{MSE}), \quad (12)$$

$$\text{MSE} = \frac{1}{W \cdot H} \sum_{i=1}^W \sum_{j=1}^H (I_{wat}(i, j) - I_{enc}(i, j))^2, \quad (13)$$

$$\text{BPP} = n_b / (W \cdot H), \quad (14)$$

where I_{wat} and I_{enc} are watermarked (embedded) cipher and (encrypted and) encoded cover images, respectively and n_b is denotes the number of embedded data bits.

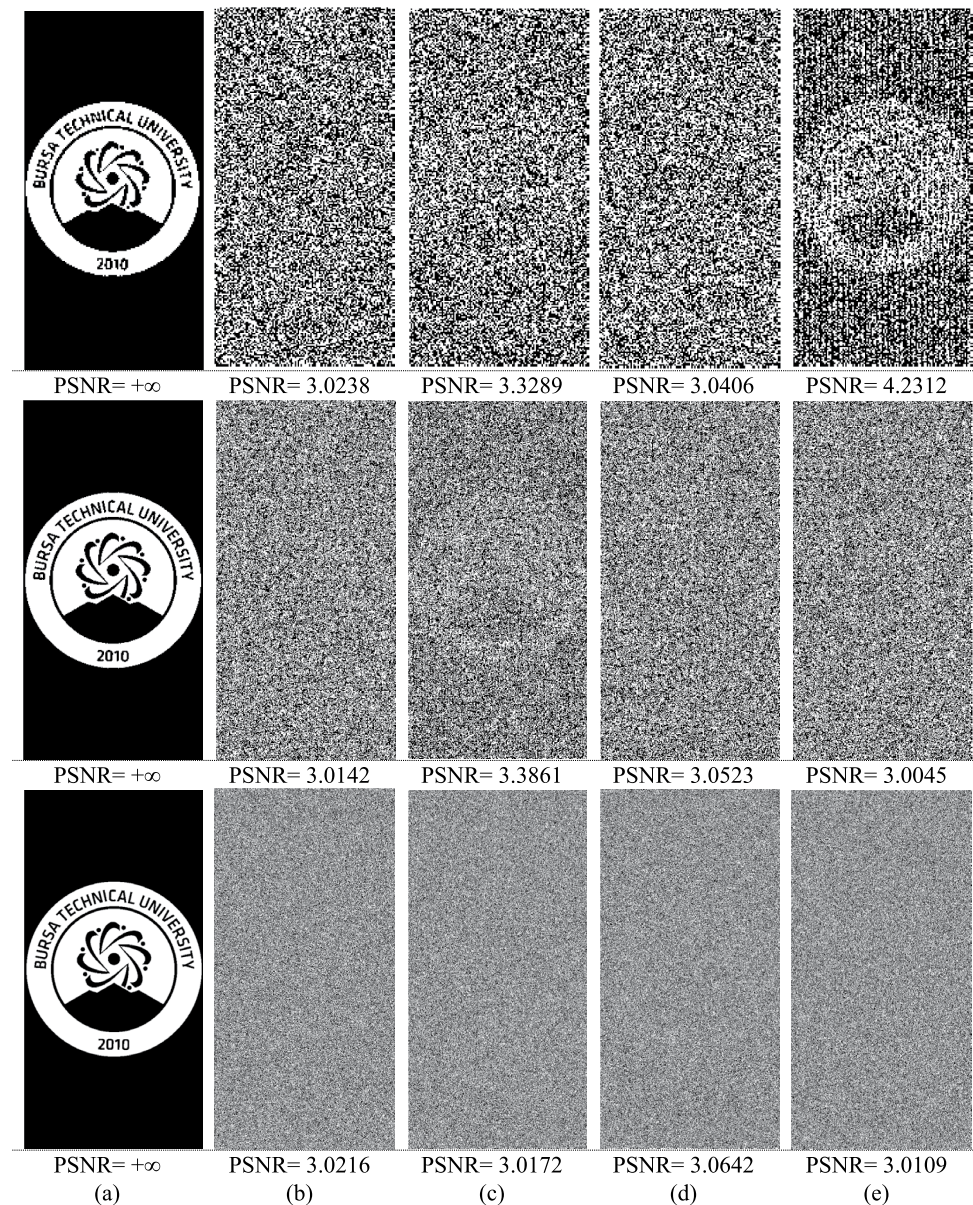
3.2.1 Reversibility analysis

One of the essential requirements of the RDH schemes is reversibility, which means that the scheme should allow for extraction of the embedded data and complete recovery of the original image in the case that there is no attack in the watermarked image [30]. Results of the reversibility analysis are presented for 9 test images in Table 4. $\text{PSNR}_{\text{rec}}^{\text{org}}$ stands for the PSNR (in dB) between recovered and original cover images and $\text{PSNR}_{\text{ext}}^{\text{emb}}$ represents the PSNR (in dB) between extracted and embedded secret images. One can easily see that the proposed scheme is fully reversible since both PSNR values are infinite for all the test images.

3.2.2 Evaluation of image recovery and data extraction results

Case 1 examines the situation where both the encryption and data-hiding keys are available. The test image “Elaine” is selected as original image and a binary image of size 512×256 , as shown in Fig. 9, is selected as secret data in this experiment. The dimensions of the secret image are

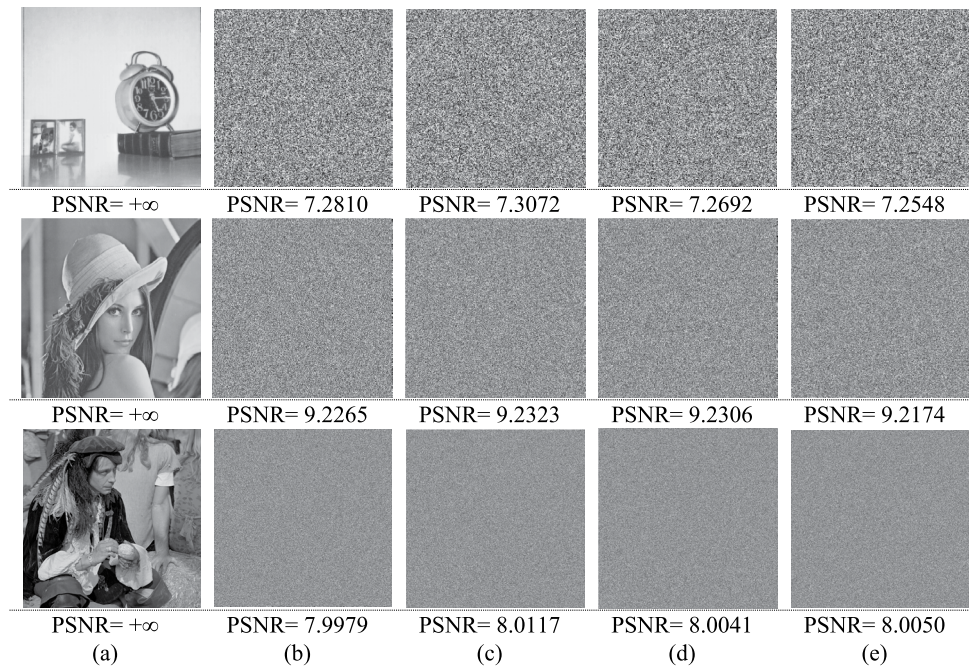
Fig. 13 Secret image results of noise attack experiments: **a** Intact, **b** Gaussian noise (0.01), **c** Speckle noise (0.2), **d** Median filtering (3×3), **e** Poisson noise



adjusted according to the varying sizes of the test image. For instance, when selecting a test image with dimensions of 256×256 , a corresponding secret image with dimensions of 256×128 is utilized. This indicates that the secret data has a length of 131,072 bits, which is half the total number of pixels in the original image. Consequently, the embedding rate is 0.5 BPP. Figure 10 shows outputs of some phases in this experiment. Original test image is shown in Fig. 10a and its encrypted counterpart is displayed in Fig. 10b. The encoded image and the watermarked image (containing the embedded data) are displayed in Fig. 10c, d, respectively. Figure 10e presents the absolute difference between the encoded image and the watermarked image. In this representation, black pixels indicate no difference in pixel values between the watermarked and encoded images, while

non-black pixels denote a non-zero difference between the two. Upon closer inspection, the non-black pixels can be observed scattered throughout the image. To quantify this effect, we computed the $\text{PSNR}_{\text{wat}}^{\text{enc}}$ metric, which measures PSNR between the watermarked and encoded (cover) images. As shown in Table 5, the $\text{PSNR}_{\text{wat}}^{\text{enc}}$ value for the test image "Elaine", along with other test images, exceeds 24 dB. This indicates that the embedding process minimally alters the encoded image, as none of the PSNR values fall below 20 dB, a threshold typically associated with poor image quality [35]. Consequently, the process of data hiding does not compromise the security integrity of the encoded cipher image. As illustrated in Fig. 10f, the decryption results of the extracted data demonstrate accurate retrieval, with no errors detected. The zero-error rate in the extraction

Fig. 14 Original image results of noise attack experiments: **a** Intact, **b** Gaussian noise (0.01), **c** Speckle noise (0.2), **d** Median filtering (3×3), **e** Poisson noise



of the hidden image data confirms that the embedded information can be retrieved in a completely lossless manner [13]. The distortion-free recovered image is presented in Fig. 10g. This experiment affirms that the proposed scheme consistently ensures reversibility, while maintaining information security, with an embedding rate of 0.5 BPP.

Case 2 addresses the experimental results when only the data-hiding key is owned by the receiver side (Fig. 11). In this experiment, with access solely to the data-hiding key, the original image cannot be reconstructed, as illustrated in Fig. 11b. The figure reveals noise-like data, indicating that no content from the cover image can be discerned from the encrypted image. However, as demonstrated in Fig. 11c, the secret image is successfully extracted and is identical to the one presented in Fig. 9. This case may exemplify a scenario in which a data-hider (for example, in a secure cloud environment) may not know the original content, yet can still embed additional data into the encrypted image [23] and subsequently extract the secret image successfully.

Case 3 analyzes the experimental results when the receiver possesses only the decryption key, as depicted in Fig. 12. In this scenario, both the recovered cover image and the extracted secret image exhibit noise-like characteristics. Consequently, without the decryption key, no meaningful information can be retrieved from the images.

3.2.3 Bit size comparison of watermarked images

A comparative analysis of the bit sizes of watermarked images produced by various data hiding methods from the literature is conducted, as each method generates

watermarked images of different sizes. To evaluate the performance of our proposed scheme, it is compared with several similar approaches [17, 28–30, 40, 41]. Table 6 presents the bit size of the watermarked images generated by the compared schemes. As observed in the table, the only scheme among those compared that maintains the same size for both the original and watermarked images is the study by Xiao et al. [28]. In contrast, the watermarked images generated by the schemes of Xiong et al. [30] and Bhardwaj and Aggarwal [29] are four times larger than the original images. This increase in size is due to the direct interpolation of the original image by a factor of 2 in these two methods. The remaining schemes, aside from this one, generate watermarked images whose sizes differ from those of the original images. The size of the watermarked images produced by the schemes of Malik et al. [17] and Zhang et al. [40] is $(\lfloor \log_2 N^2 \rfloor + 1)/8$ times larger than the original images. This is due to the use of the Paillier encryption system, where the equation $N = p \times q$ is employed, with p and q being large prime numbers [30]. For instance, in Malik et al.'s study, a minimum of $N = 77$ is reported, resulting approximately in a 1.7-fold increase in the size of the watermarked images. The size of the watermarked image generated by the scheme of Bhardwaj [20] is $(\lfloor \log_2 N^2 \rfloor + 1)/2$ times larger than the original images, which is even worse. Our scheme results in only a one-bit increase in the factor of $W \times H$ for the watermarked image, increasing it from 8 to 9 compared to the original image. Since the embedding technique employed in the proposed study is based on the CRT, it results in minimal expansion in the size of the watermarked image.

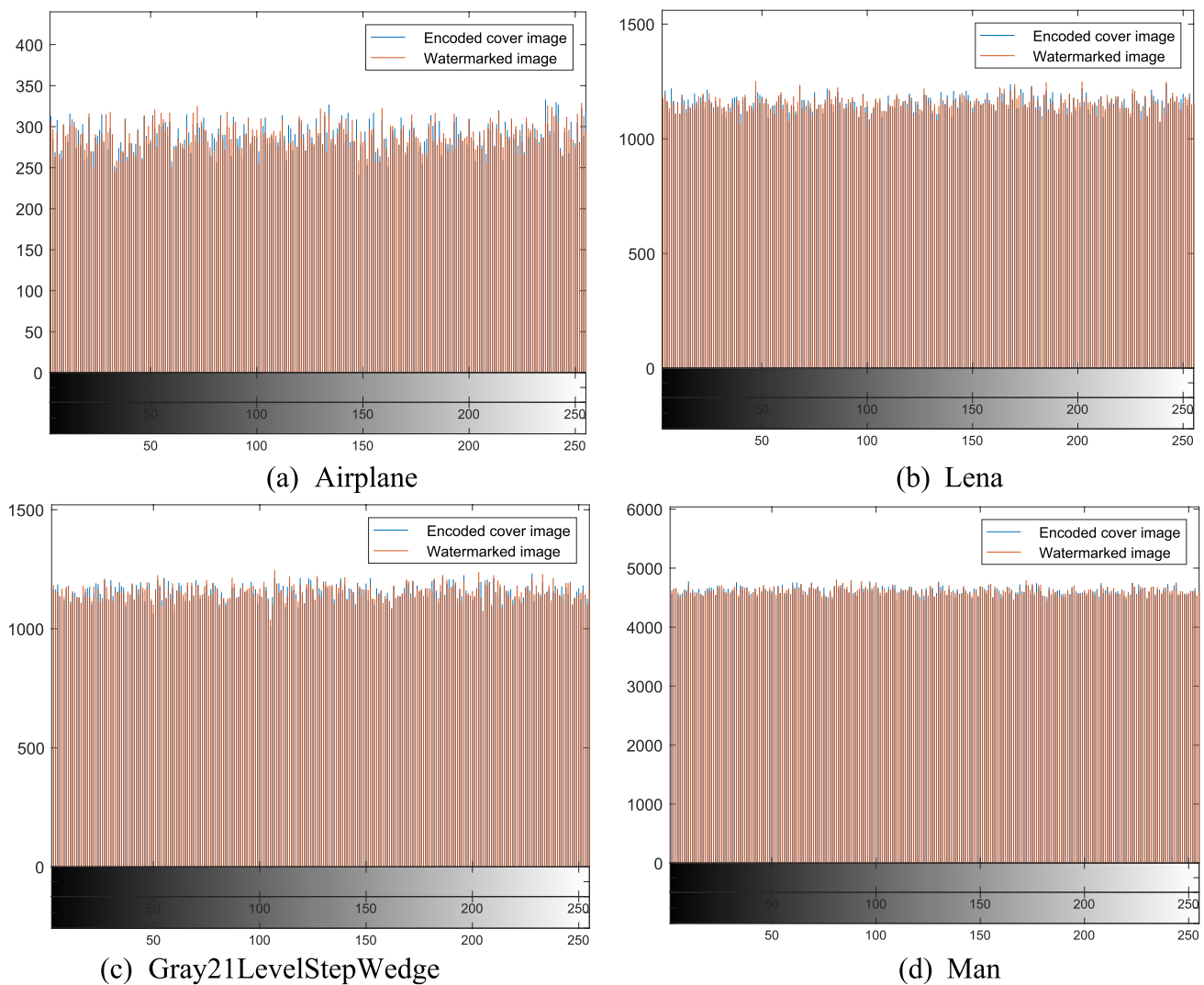


Fig. 15 Histogram comparisons between encoded cover image and watermarked image: **a** Airplane, **b** Lena, **c** Gray21LevelStepWedge, **d** Man

Table 8 Comparative analysis of the performance of the proposed scheme and related studies

Schemes	Average BPP	PSNR _{wat} ^{enc}	WOSR	Fixed EC
Xiao et al. [28]	0.49	N/A	1	No
Xiong et al. [30]	2.25	~41 dB	4	Yes
Bhardwaj and Aggarwal [29]	2.24	~42 dB	4	Yes
Malik et al. [17]	0.48	N/A	$(\lfloor \log_2 N^2 \rfloor + 1)/8$	No
Bhardwaj [20]	3	N/A	$(\lfloor \log_2 N^2 \rfloor + 1)/2$	Yes
Proposed scheme	0.5	~24 dB	1.125	Yes

3.2.4 Key characteristics of evaluated schemes

An overview of the key characteristics of the evaluated schemes is presented in Table 7. Notably, all of the compared methods, with the exception of the one presented in [40], successfully extract the embedded secret data

accurately and fully recover the original image without any loss. Some of these schemes [17, 20, 28] implement a non-separable extraction process, meaning the secret data cannot be retrieved without the decryption key, and the original (host/cover) image cannot be recovered without the data hiding key. In contrast, other schemes [29, 30, 40] employ a (fully) separable extraction process, which offers greater flexibility by allowing either the original image or the secret data to be recovered independently, without the need for both keys.

The extraction stage of our proposed scheme falls between these two approaches and is described as "semi-separable." As demonstrated in Fig. 11, if only the data hiding key is available, our scheme allows for the extraction of the secret image without requiring the decryption key. On the other hand, if only the decryption key is available, as illustrated in Fig. 12, neither the secret data nor the original host image can be recovered—both appear as noise-like

Table 9 Key steps in the embedding process and their computational complexity

Key steps	Complexity (Big- O)
Step 1–3	$O(3 \times W \times H)$
Step 4	$O(4 \times W \times H)$
Step 5	$O(8 \times W \times H)$
Step 6–8	$O(5 \times W \times H)$

Table 10 Computational complexity comparison of embedding process across several schemes

Schemes	Complexity (Big- O)
Wang et al. [13]	$O(4 \times W \times H)$
Xiong et al. [30]	$O(9 \times W \times H)$
Ankur et al. [41]	$O((W \times H) \log_2(W \times H))$
Wu et al. [42] ^a	$O(N \times W \times H)$
Zheng et al. [43] ^b	$O(2^B \times W \times H)$
Proposed scheme	$O(20 \times W \times H)$

^a N is a number of coefficient used in the compared study [42]

^b B is bit value used in the compared study [43]

Table 11 Average execution times of encryption and embedding processes for test images of various sizes

Test images	Execution time of encryption process (in seconds)	Execution time of embedding process (in seconds)
Airplane	0.0389	0.0386
Lena	0.1606	0.1699
Male	0.6474	0.6916

Table 12 Comparison of average execution times (in seconds) across different schemes

Schemes	Encryption process	Embedding process
Xiao et al. [28]	0.0008	1.3205
Bhardwaj and Aggarwal [29]	0.0009	1.1438
Xiong et al. [30]	0.9449	0.1677
Proposed scheme	0.6474	0.6916

outputs. These scenarios are discussed in detail in the previous Sect. 3.2.2. Consequently, the behavior of our scheme in such conditions supports the characterization of its extraction process as semi-separable, as summarized in Table 7.

3.2.5 Authentication analysis

To assess authentication performance, secret images of various sizes, including the 512×256 version shown in Fig. 9, were embedded. The watermarked images were then subjected to various noise attacks, such as Gaussian noise, speckle noise, median filtering, and Poisson noise. Figure 13 presents the extracted secret images following the

attacks, while Fig. 14 depicts the recovered original (cover) images post-attack. As observed from the figures, the data extraction method successfully recovers the embedded image in its original form when the watermarked image is not exposed to common noise attacks. However, after typical attacks, the recovered image becomes unrecognizable to the human eye [30], as demonstrated by the low PSNR values (in dB) displayed below each subfigure. As noted in [35], PSNR values below 20 dB indicate poor image quality, and all PSNR values obtained in the experiment were lower than 10 dB for both the extracted and original images. These results demonstrate that the data hiding technique disseminates the secret data across the entirety of the cover image and effectively ensures a high level of scrambling.

3.2.6 Histogram analysis

By visually analyzing the histograms of both the encoded cover image and the corresponding watermarked image, their differences can usually be observed [30]. A smaller difference between the histograms suggests that the embedded data has minimally impacted the cover image, indicating better performance of the data embedding technique. Figure 15 presents the experimental results for four test images, leading to two main observations. First, both histograms display a uniform distribution, aligning with the expected pattern for encrypted images. Second, the histograms of the watermarked and encoded cover images are highly similar. These results highlight the effectiveness of the proposed data embedding approach.

3.2.7 Embedding capacity analysis

The embedding capacity (EC), often referred to as the embedding rate, is a critical metric for evaluating the effectiveness of data embedding techniques. To quantify the EC of the proposed method, we use the BPP metric, as defined in Eq. (14). In general, for a given level of watermarked image quality (measured by PSNR), a higher embedding rate corresponds to better performance [30]. In the case of our data embedding approach, which is based on the CRT, two encrypted pixels are used to conceal one bit of secret data, resulting in a consistent (or fixed) EC of 0.5 BPP. This embedding rate remains constant, irrespective of the complexity of the original images. Specifically, our method guarantees an embedding rate of 0.5 BPP across all test images, which exhibit entropy values ranging approximately from 4.4 to 7.6, as shown in Fig. 6. The EC is attained with a watermarked image quality $\text{PSNR}_{\text{wat}}^{\text{enc}}$ exceeding 24 dB, as indicated in Table 5. As previously noted, $\text{PSNR}_{\text{wat}}^{\text{enc}}$ refers

to the peak signal-to-noise ratio between the watermarked and encoded (cover) images, and it serves as a key indicator for assessing watermarked image quality. This metric is crucial for evaluating the overall performance of the data embedding method. Typically, for a fixed embedding rate, a higher PSNR value reflects improved performance of the embedding process, signifying better preservation of image quality after embedding. Besides, the ratio of the watermarked image size to the original image size can be referred to as the WOSR. This metric quantifies the proportional increase in size due to embedding and is an important factor in assessing the efficiency and impact of the data embedding process. A WOSR close to 1 indicates minimal size alteration, signifying efficient embedding with little overhead. This metric for the comparative studies listed in Table 8 can be derived from the data presented in Table 6.

Table 8 presents a comparative analysis of the performance of the proposed scheme and several related studies from the literature. In comparing the data embedding schemes listed, several key metrics stand out, notably the EC (measured in BPP), the watermarked image quality ($\text{PSNR}_{\text{wat}}^{\text{enc}}$), WOSR, and whether the EC is fixed. Our proposed scheme achieves an consistent EC of 0.5 BPP with a $\text{PSNR}_{\text{wat}}^{\text{enc}}$ of approximately 24 dB and a WOSR of 1.125. This fixed EC places it slightly above Xiao et al.'s scheme [28], and Malik et al.'s [17] schemes, both of which achieve BPP values of 0.49 and 0.48, respectively, without providing specific PSNR data. Among the compared schemes, only Xiao et al.'s study [28] achieves a WOSR of 1, indicating that their embedding approach does not result in any expansion of the image size. As discussed in Sect. 3.2.3, Malik et al.'s scheme reports a minimum value of $N=77$, resulting in an approximate WOSR of 1.7 based on the equation $(\lfloor \log_2 N^2 \rfloor + 1)/8$. This indicates that their scheme causes approximately a 70% increase in the size of the transmitted data. Unfortunately, both Malik et al.'s [17] and Xiao et al.'s [28] schemes lack a fixed embedding capacity, as their embedding rate varies depending on the entropy of the original (cover) images used. In contrast, the schemes by Xiong et al. [30] and Bhardwaj and Aggarwal [29] attain significantly higher BPP values, around 2.25, coupled with superior PSNR values near 41–42 dB, reflecting a better preservation of watermarked image quality. However, these methods result in higher WOSR values, indicating a greater increase in image size. Bhardwaj's [20] method, which achieves an even higher BPP of 3, does not report PSNR values but also features an elevated WOSR. Notably, the proposed scheme balances moderate EC with fixed image quality metrics, ensuring minimal size distortion while

maintaining reasonable PSNR values. This makes it a competitive choice for applications requiring consistent image quality and efficient data embedding.

3.2.8 Computational complexity analysis

The proposed scheme comprises two primary stages: image encryption and data embedding. The encryption stage functions as an independent cipher; in this study, a recently developed chaos-based cryptosystem by the author has been employed. Consequently, a complexity analysis of the encryption stage is not included. However, the computational complexity of key steps in the embedding process [30] is outlined in Table 9. Accordingly, the overall computational complexity of the scheme is approximately $O(20 \times W \times H)$. Table 10 provides a computational complexity comparison for the embedding process across several schemes [13, 30, 41–43]. However, as few studies report the complexity of their embedding schemes, the comparison is limited. Instead, many authors report only the execution time of their schemes. Although measuring algorithm execution time on time-shared operating systems may not accurately reflect true execution times, we nonetheless follow this approach, reporting run times alongside computational complexity.

In this experiment, the average execution time of the proposed scheme was calculated over 100 executions. Table 11 presents the average running times for both the encryption and embedding processes across test images of varying sizes. According to the data presented in the table, the embedding process for a test image (namely “Male”) with dimensions of 1024×1024 completes in under one second. Additionally, the combined execution time of the encryption and embedding processes is less than 2 s. Although the proposed scheme has higher computational complexity compared to some studies, it addresses gaps in existing schemes related to vulnerability to malicious attacks and overhead from image size expansion for storage and transmission.

Moreover, Table 12 presents a comparison of average execution times (in seconds) across different schemes. For our proposed method, the average was calculated over 100 runs. The execution times for other schemes [28–30] were referenced from Xiong et al. [30]. However, since most state-of-the-art (SOTA) methods report timings under varying hardware and implementation conditions, direct runtime comparisons may be unreliable. Therefore, we primarily rely on the computational complexity analysis provided in Table 10.

3.3 Limitations

While the proposed RDHEI scheme achieves promising outcomes in stable-capacity reversible data hiding, it is essential to recognize the design-related trade-offs inherent to its approach.

1. *Limited embedding capacity flexibility*: While the proposed scheme maintains a stable (or consistent) embedding capacity of 0.5 BPP regardless of image complexity, it lacks the flexibility to dynamically adjust the embedding rate based on application-specific requirements, which may be advantageous in scenarios with varying payload sizes or where adaptive capacity control is critical.
2. *Computational overhead in pixel fusion and CRT operations*: The use of CRT-based pixel fusion, although secure and reversible, introduces additional computational complexity due to modular arithmetic operations, which may lead to increased processing time compared to simpler embedding schemes—especially in resource-constrained environments.
3. *Semi-separability restriction in certain recovery scenarios*: The proposed method achieves semi-separability by allowing secret data extraction without the decryption key; however, it does not support full separability, meaning that independent recovery of the original image without the data-hiding key remains infeasible. This may limit its applicability in systems requiring greater extraction flexibility or partial access control.

4 Conclusion

This paper proposes a new RDHEI scheme that combines chaos theory with the Chinese Remaining Theorem (CRT) to address fundamental challenges in secure and reversible data embedding. By leveraging a chaos-based symmetric cipher and an innovative pixel fusion strategy, the proposed method achieves a consistent embedding capacity (EC) of 0.5 BPP, independent of host image complexity, enhancing both efficiency and robustness. Our approach, based on CRT, allows for the secure and reversible embedding of one bit of secret data using two encrypted pixels without relying on spatial correlation, thus broadening the scheme's applicability across diverse media types. Notably, the method achieves semi-separability, enabling the extraction of the embedded data with only the data-hiding key, eliminating the need for the decryption key, which adds flexibility to the process. Experimental results validate the effectiveness of

the proposed method, demonstrating a minimal increase in watermarked-to-original size ratio (WOSR) and its ability to withstand common cryptographic attacks. In comparison to existing state-of-the-art techniques, our scheme stands out in embedding stability, security, and reversibility, showcasing its potential as a viable and practical RDHEI solution for secure data hiding in encrypted images. Future research may focus on optimizing multicore systems by leveraging the algorithm's loop-level parallelism to enhance EC, reduce running time, and extend the method to other media formats.

Acknowledgements This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Author contributions Entire work in the manuscript was done by the corresponding author.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK).

Data availability Data links is provided within the manuscript.

Code availability Code repository name: <https://github.com/eyavuz341/RDH>

Declarations

Competing interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Yavuz E (2019) A novel chaotic image encryption algorithm based on content-sensitive dynamic function switching scheme. *Opt Laser Technol* 114:224–239. <https://doi.org/10.1016/j.optlastec.2019.01.043>
2. Niyat AY, Moattar MH, Torshiz MN (2017) Color image encryption based on hybrid hyper-chaotic system and cellular automata. *Opt Lasers Eng* 90:225–237. <https://doi.org/10.1016/j.optlaseng.2016.10.019>

3. Cox IJ, Miller ML, Bloom JA, Fridrich J, Kalker T (2007) Digital watermarking and steganography. Elsevier, Amsterdam
4. Makhdoom I, Abolhasan M, Lipman J (2022) A comprehensive survey of covert communication techniques, limitations and future challenges. *Comput Secur* 120:102784. <https://doi.org/10.1016/j.cose.2022.102784>
5. Fridrich J (2009) Steganography in digital media: principles, algorithms, and applications. Cambridge University Press, Cambridge
6. Tiwari A, Sharma M, Tamrakar RK (2017) Watermarking based image authentication and tamper detection algorithm using vector quantization approach. *Int J Electron Commun* 78:114–123. <https://doi.org/10.1016/j.aecue.2017.05.027>
7. Yuan Z, Zhang X, Wang Z, Yin Z (2024) Semi-fragile neural network watermarking for content authentication and tampering localization. *Expert Syst Appl* 236:121315. <https://doi.org/10.1016/j.eswa.2023.121315>
8. Garfinkel SL (2010) Digital forensics research: the next 10 years. *Digit Investig* 7:64–S73
9. AboSamra KM, AbdelHafez AA, Assassa GM, Mursi MF (2017) A practical, secure, and auditable e-voting system. *J Inf Secur Appl* 36:69–89. <https://doi.org/10.1016/j.jisa.2017.08.002>
10. Baseri Y, Hafid A, Firoozjaei MD, Cherkaoui S, Ray I (2024) Statistical privacy protection for secure data access control in cloud. *J Inf Secur Appl* 84:103823. <https://doi.org/10.1016/j.jisa.2024.103823>
11. Sicari S, Rizzardi A, Grieco LA, Coen-Porisini A (2015) Security, privacy and trust in Internet of Things: The road ahead. *Comput Netw* 76:146–164. <https://doi.org/10.1016/j.comnet.2014.11.008>
12. Rai AK, Om H, Chand S, Agarwal S (2023) Reversible data hiding in encrypted image using two-pass pixel value ordering. *J Inf Secur Appl* 76:103545. <https://doi.org/10.1016/j.jisa.2023.103545>
13. Wang Y, Xiong G, He W (2023) High-capacity reversible data hiding in encrypted images based on pixel-value-ordering and histogram shifting. *Expert Syst Appl* 211:118600. <https://doi.org/10.1016/j.eswa.2022.118600>
14. He W, Xiong G, Weng S, Cai Z, Wang Y (2018) Reversible data hiding using multi-pass pixel-value-ordering and pairwise prediction-error expansion. *Inf Sci* 467:784–799. <https://doi.org/10.1016/j.ins.2018.04.088>
15. Ou B, Li X, Wang J (2016) High-fidelity reversible data hiding based on pixel-value-ordering and pairwise prediction-error expansion. *J Vis Commun Image Represent* 39:12–23. <https://doi.org/10.1016/j.jvcir.2016.05.005>
16. Shi YQ, Li X, Zhang X, Wu HT, Ma B (2016) Reversible data hiding: advances in the past two decades. *IEEE Access* 4:3210–3237. <https://doi.org/10.1109/ACCESS.2016.2573308>
17. Malik A, Wang H, Chen T, Yang T, Khan AN, Wu H, Chen Y, Hu Y (2019) Reversible data hiding in homomorphically encrypted image using interpolation technique. *J Inf Secur Appl* 48:102374. <https://doi.org/10.1016/j.jisa.2019.102374>
18. Xiong L, Shi Y (2018) On the privacy-preserving outsourcing scheme of reversible data hiding over encrypted image data in cloud computing. *Comput. Mater. Contin.* 55:523–539. <https://doi.org/10.3970/cmcc.2018.01791>
19. Kumar R, Chand S, Singh S (2018) An improved histogram-shifting-imitated reversible data hiding based on HVS characteristics. *Multimed Tools Appl* 77:13445–13457. <https://doi.org/10.1007/s11042-017-4960-y>
20. Bhardwaj R (2021) An enhanced reversible patient data hiding algorithm for e-healthcare. *Biomed Signal Process Control* 64:102276. <https://doi.org/10.1016/j.bspc.2020.102276>
21. Li X, Li J, Li B, Yang B (2013) High-fidelity reversible data hiding scheme based on pixel-value-ordering and prediction-error expansion. *Signal Process* 93:198–205. <https://doi.org/10.1016/j.sigpro.2012.07.025>
22. Kaur G, Singh S, Rani R, Kumar R (2021) A comprehensive study of reversible data hiding (RDH) schemes based on pixel value ordering (PVO). *Arch Comput Methods Eng* 28:3517–3568. <https://doi.org/10.1007/s11831-020-09512-3>
23. Zhang X (2011) Reversible data hiding in encrypted image. *IEEE Signal Process Lett* 18:255–258. <https://doi.org/10.1109/LSP.2011.2114651>
24. Zhang X (2011) Separable reversible data hiding in encrypted image. *IEEE Trans Inf Forensics Secur* 7:826–832. <https://doi.org/10.1109/TIFS.2011.2176120>
25. Karim MSA, Wong K (2014) Universal data embedding in encrypted domain. *Signal Process* 94:174–182. <https://doi.org/10.1016/j.sigpro.2013.06.014>
26. Zhang X, Qian Z, Feng G, Ren Y (2014) Efficient reversible data hiding in encrypted images. *J Vis Commun Image Represent* 25:322–328. <https://doi.org/10.1016/j.jvcir.2013.11.008>
27. Liao X, Shu C (2015) Reversible data hiding in encrypted images based on absolute mean difference of multiple neighboring pixels. *J Vis Commun Image Represent* 28:21–27. <https://doi.org/10.1016/j.jvcir.2014.12.007>
28. Xiao D, Wang Y, Tao X, Bai S (2017) High-payload completely reversible data hiding in encrypted images by an interpolation technique. *Front Inf Technol Electron Eng* 18:12. <https://doi.org/10.1631/FITEE.1601067>
29. Bhardwaj R, Aggarwal A (2021) An enhanced separable reversible and secure patient data hiding algorithm for telemedicine applications. *Expert Syst Appl* 186:115721. <https://doi.org/10.1016/j.eswa.2021.115721>
30. Xiong X, Zhong S, Lu Y (2024) Separable and reversible data hiding scheme for medical images using modified Logistic and interpolation. *Biomed Signal Process Control* 87:105521. <https://doi.org/10.1016/j.bspc.2023.105521>
31. Zhu H, Zhao C, Zhang X (2013) A novel image encryption-compression scheme using hyper-chaos and Chinese remainder theorem. *Signal Process Image Commun* 28:670–680. <https://doi.org/10.1016/j.image.2013.02.004>
32. SIPI Image Database, University of Southern California Signal and Image Processing Institute. <http://sipi.usc.edu/database/>. Accessed 2 Mar 2024.
33. Baker GL, Gollub JP (1996) Chaotic dynamics: an introduction, 2nd edn. Cambridge University Press, Cambridge
34. Wu Y, Noonan JP, Agaian S (2011) NPCR and UACI randomness tests for image encryption. *Cyber J* 1(2):31–38
35. Yavuz E (2021) A new parallel processing architecture for accelerating image encryption based on chaos. *J Inf Secur Appl* 63:103056. <https://doi.org/10.1016/j.jisa.2021.103056>
36. Wu Y, Zhou Y, Saveriades G, Agaian S, Noonan JP, Natarajan P (2013) Local Shannon entropy measure with statistical tests for image randomness. *Inf Sci* 222:323–342. <https://doi.org/10.1016/j.ins.2012.07.049>
37. Liu D, Zhang W, Yu H, Zhu ZL (2018) An image encryption scheme using self-adaptive selective permutation and inter-intra-block feedback diffusion. *Signal Process* 151:130–143. <https://doi.org/10.1016/j.sigpro.2018.05.008>
38. Alawida M, Samsudin A, Teh JS, Alkhalwaldeh RS (2019) A new hybrid digital chaotic system with applications in image encryption. *Signal Process* 160:45–58. <https://doi.org/10.1016/j.sigpro.2019.02.016>

39. Yavuz E, Yazıcı R, Kasapbaşı MC, Yamaç E (2016) A chaos-based image encryption algorithm with simple logical functions. *Comput Electr Eng* 54:471–483. <https://doi.org/10.1016/j.compeleceng.2015.11.008>
40. Zhang X, Long J, Wang Z, Cheng H (2016) Lossless and reversible data hiding in encrypted images with public-key cryptography. *IEEE Trans Circuits Syst Video Technol* 26:1622–1631. <https://doi.org/10.1109/TCSVT.2015.2433194>
41. Kumar AR, Sharma AK (2025) Reversible data hiding in encrypted image using bit-plane based label-map encoding with optimal block size. *J Inf Secur Appl* 90:104005. <https://doi.org/10.1016/j.jisa.2025.104005>
42. Wu HT, Cheung YM, Tian Z, Liu D, Luo X, Hu J (2024) Lossless data hiding in NTRU cryptosystem by polynomial encoding and modulation. *IEEE Trans Inf Forensics Secur* 19:3719–3732. <https://doi.org/10.1109/TIFS.2024.3362592>
43. Zheng S, Wang Y, Hu D (2019) Lossless data hiding based on homomorphic cryptosystem. *IEEE Trans Dependable Secure Comput* 18(2):692–705. <https://doi.org/10.1109/TDSC.2019.2913422>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.